

Human review packet: GGRS26CombatingGerrymanderingRCV

Generated from byte-pinned source excerpts, the expanded PaperInterface specifications, and hash-bound raw-source-to-Spec screening records.

Paper: GGRS26CombatingGerrymanderingRCV
Paper id: GGRS26CombatingGerrymanderingRCV
Source version: arXiv:2107.07083 source bundle and byte-pinned cited publication transcript
Source record: audit/paper_statement_map.json
Rows: 2
Generated: 2026-09-06
Source URL: [official source](#)

How to use this packet

For each source claim, compare the exact source excerpts with the expanded PaperInterface specification. Mark up this PDF or fill the blank reviewer box in the TeX source; return the annotated file for reconciliation into the paper-local human-review log.

Open the interactive dashboard

The interactive dashboard is an optional alternative to using this PDF; it presents the same review surface rather than an additional review requirement. After forking and cloning (or pulling) AppliedModelingLib, run the following from the repository root. The cache command is needed only the first time in a new clone.

```
cd <your-repository-checkout>
lake exe cache get
python3 scripts/review_dashboard.py --paper GGRS26CombatingGerrymanderingRCV --serve
```

Open <http://127.0.0.1:8765/> in a browser and keep that terminal running while reviewing. The dashboard records saved annotations in this paper's local reviewer-owned review record.

Contents

- [Governing Lean declarations \(32; supporting context\)](#)
- [Paper-specific semantic prerequisites \(24; not paper claims\)](#)
 - [GGRS26CombatingGerrymanderingRCV.PaperSourceSTVState](#)
 - [GGRS26CombatingGerrymanderingRCV.PaperThieleWeights](#)
 - [GGRS26CombatingGerrymanderingRCV.paper_candidate_ranked_before](#)
 - [GGRS26CombatingGerrymanderingRCV.paper_complete_solid_coalition_rankings](#)
 - [GGRS26CombatingGerrymanderingRCV.paper_first_active_candidate](#)
 - [GGRS26CombatingGerrymanderingRCV.paper_thiele_committee_score](#)
 - [GGRS26CombatingGerrymanderingRCV.paper_thiele_fixed_size_selected_committee](#)
 - [GGRS26CombatingGerrymanderingRCV.paper_pav_fixed_size_selected_committee](#)
 - [GGRS26CombatingGerrymanderingRCV.paper_thiele_party_min_argmax](#)
 - [GGRS26CombatingGerrymanderingRCV.paper_thiele_party_republican_selection](#)
 - [GGRS26CombatingGerrymanderingRCV.paper_thiele_party_republican_seat_count](#)
 - [GGRS26CombatingGerrymanderingRCV.paper_pav_selected_seat_count](#)
 - [GGRS26CombatingGerrymanderingRCV.paper_pav_selected_seat_count_spec](#)
 - [GGRS26CombatingGerrymanderingRCV.paper_source_stv_execution_terminal](#)
 - [GGRS26CombatingGerrymanderingRCV.paper_source_stv_tally](#)
 - [GGRS26CombatingGerrymanderingRCV.paper_source_stv_d_favoring_transition](#)
 - [GGRS26CombatingGerrymanderingRCV.paper_source_surplus_transfer_policy](#)
 - [GGRS26CombatingGerrymanderingRCV.paper_source_stv_rule_execution](#)
 - [GGRS26CombatingGerrymanderingRCV.paper_thiele_party_democratic_seat_count](#)
 - [GGRS26CombatingGerrymanderingRCV.paper_thiele_party_republican_seat_count_spec](#)
 - [GGRS26CombatingGerrymanderingRCV.paper_thiele_party_selected_seat_counts](#)
 - [GGRS26CombatingGerrymanderingRCV.paper_thiele_party_selected_seat_counts_add](#)
 - [GGRS26CombatingGerrymanderingRCV.paper_thiele_party_selected_seat_counts_spec](#)
 - [GGRS26CombatingGerrymanderingRCV.paper_two_party_voter_model](#)
- [Source claims \(2\)](#)
 - [paper_lemma_c1_pav_selector_eq_unique_integer_intervalSpec](#)
 - [paper_proposition1_source_selected_stv_and_pavSpec](#)

Governing Lean declarations

These exact graph-bound declaration bodies are retained by the displayed specifications or reviewed prerequisites. They are supporting context and do not add source-result rows or semantic judgments.

AppliedModelingLib.SocialChoice.Voting.ApprovalBallot

Declaration location: reusable library

Lean declaration body

```
type:
Type u_1 → Type u_1

value:
fun (Candidate : Type u_1) => Finset Candidate
```

AppliedModelingLib.SocialChoice.Voting.Ballot

Declaration location: reusable library

Lean declaration body

```
type:
Type u_1 → Type u_1

value:
fun (Candidate : Type u_1) => List Candidate
```

AppliedModelingLib.SocialChoice.Voting.Ballot.Valid

Declaration location: reusable library

Lean declaration body

```
type:
{Candidate : Type u_1} → AppliedModelingLib.SocialChoice.Voting.Ballot Candidate → Prop

value:
fun {Candidate : Type u_1} (ballot : AppliedModelingLib.SocialChoice.Voting.Ballot Candidate) => List.Nodup ballot
```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Voting.Ballot](#)

AppliedModelingLib.SocialChoice.Voting.Ballot.nextActive

Declaration location: reusable library

Lean declaration body

```
type:
{Candidate : Type u_1} →
[DecidableEq Candidate] →
AppliedModelingLib.SocialChoice.Voting.Ballot Candidate → Finset Candidate → Option Candidate

value:
fun {Candidate : Type u_1} [DecidableEq Candidate] (ballot : AppliedModelingLib.SocialChoice.Voting.Ballot Candidate)
(active : Finset Candidate) =>
List.brecOn ballot (AppliedModelingLib.SocialChoice.Voting.Ballot.nextActive._f active)
```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Voting.Ballot](#)

AppliedModelingLib.SocialChoice.Voting.Ballot.activeSupport

Declaration location: reusable library

Lean declaration body

```
type:
{Voter : Type u_1} →
{Candidate : Type u_2} →
[DecidableEq Candidate] →
Finset Voter →
(Voter → AppliedModelingLib.SocialChoice.Voting.Ballot Candidate) → Finset Candidate → Candidate → Finset Voter

value:
fun {Voter : Type u_1} {Candidate : Type u_2} [DecidableEq Candidate] (voters : Finset Voter)
(ballots : Voter → AppliedModelingLib.SocialChoice.Voting.Ballot Candidate) (active : Finset Candidate)
(candidate : Candidate) =>
{voter ∈ voters | (ballots voter).nextActive active = some candidate}
```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Voting.Ballot](#)
- [AppliedModelingLib.SocialChoice.Voting.Ballot.nextActive](#)

AppliedModelingLib.SocialChoice.Voting.IsMinArgmaxOn

Declaration location: reusable library

Lean declaration body

```
type:
( $\mathbb{N} \rightarrow \mathbb{R}$ ) →  $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \text{Prop}$ 

value:
fun (score :  $\mathbb{N} \rightarrow \mathbb{R}$ ) (choice upper :  $\mathbb{N}$ ) =>
choice ≤ upper ∧
(∀ candidate ≤ upper, score candidate ≤ score choice) ∧
∀ candidate ≤ upper, score candidate = score choice → choice ≤ candidate
```

AppliedModelingLib.SocialChoice.Voting.STVQuota

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$ 

value:
fun (seats voters :  $\mathbb{N}$ ) => voters / (seats + 1) + 1
```

AppliedModelingLib.SocialChoice.Voting.approvedCommitteeCount

Declaration location: reusable library

Lean declaration body

```
type:
{Candidate : Type u_1} → [DecidableEq Candidate] → Finset Candidate → Finset Candidate →  $\mathbb{N}$ 

value:
fun {Candidate : Type u_1} [DecidableEq Candidate] (committee ballot : Finset Candidate) => (committee n ballot).card
```

AppliedModelingLib.SocialChoice.Voting.pavWeight

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{N} \rightarrow \mathbb{R}$ 

value:
fun (approved :  $\mathbb{N}$ ) => if approved = 0 then 0 else (↑ approved)-1
```

AppliedModelingLib.SocialChoice.Voting.pavHarmonicSum

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{N} \rightarrow \mathbb{R}$ 

value:
fun (a :  $\mathbb{N}$ ) => Nat.brecOn a AppliedModelingLib.SocialChoice.Voting.pavHarmonicSum._f
```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Voting.pavWeight](#)

AppliedModelingLib.SocialChoice.Voting.roundedSeatShare

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{N} \rightarrow \mathbb{R} \rightarrow \mathbb{N} \rightarrow \text{Prop}$ 

value:
fun (seatCount :  $\mathbb{N}$ ) (partyShare :  $\mathbb{R}$ ) (seats :  $\mathbb{N}$ ) =>
  seatCount = [partyShare *  $\uparrow$ seats]+ v seatCount = [partyShare *  $\uparrow$ seats]+
```

AppliedModelingLib.SocialChoice.Voting.thieleScore

Declaration location: reusable library

Lean declaration body

```
type:
{Candidate : Type u_1} →
{Score : Type u_2} →
[DecidableEq Candidate] →
[AddMonoid Score] →
( $\mathbb{N} \rightarrow \text{Score}$ ) → Finset Candidate → List (AppliedModelingLib.SocialChoice.Voting.ApprovalBallot Candidate) → Score

value:
fun {Candidate : Type u_1} {Score : Type u_2} [DecidableEq Candidate] [AddMonoid Score] (weight :  $\mathbb{N} \rightarrow \text{Score}$ )
  (committee : Finset Candidate) (profile : List (AppliedModelingLib.SocialChoice.Voting.ApprovalBallot Candidate)) =>
  (List.map
    (fun (ballot : Finset Candidate) =>
      weight (AppliedModelingLib.SocialChoice.Voting.approvedCommitteeCount committee ballot))
    profile).sum
```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Voting.ApprovalBallot](#)
- [AppliedModelingLib.SocialChoice.Voting.approvedCommitteeCount](#)

GGRS26CombattingGerrymanderingRCV.BallotRoutedSTVState

Declaration location: paper-local

Lean declaration body

```
field active:
{Voter : Type u_1} →
{Candidate : Type u_2} →
[inst : DecidableEq Voter] →
[inst_1 : DecidableEq Candidate] →
{voters : Finset Voter} →
{initialCandidates : Finset Candidate} →
GGRS26CombattingGerrymanderingRCV.BallotRoutedSTVState voters initialCandidates → Finset Candidate

field elected:
{Voter : Type u_1} →
{Candidate : Type u_2} →
[inst : DecidableEq Voter] →
[inst_1 : DecidableEq Candidate] →
{voters : Finset Voter} →
{initialCandidates : Finset Candidate} →
GGRS26CombattingGerrymanderingRCV.BallotRoutedSTVState voters initialCandidates → Finset Candidate

field weight:
{Voter : Type u_1} →
{Candidate : Type u_2} →
[inst : DecidableEq Voter] →
[inst_1 : DecidableEq Candidate] →
```

```

{voters : Finset Voter} →
{initialCandidates : Finset Candidate} →
  GGRS26CombattingGerrymanderingRCV.BallotRoutedSTVState voters initialCandidates → Voter → ℝ

field active_subset_initial:
∀ {Voter : Type u_1} {Candidate : Type u_2} [inst : DecidableEq Voter] [inst_1 : DecidableEq Candidate]
  {voters : Finset Voter} {initialCandidates : Finset Candidate}
  (self : GGRS26CombattingGerrymanderingRCV.BallotRoutedSTVState voters initialCandidates),
  self.active ⊆ initialCandidates

field elected_subset_initial:
∀ {Voter : Type u_1} {Candidate : Type u_2} [inst : DecidableEq Voter] [inst_1 : DecidableEq Candidate]
  {voters : Finset Voter} {initialCandidates : Finset Candidate}
  (self : GGRS26CombattingGerrymanderingRCV.BallotRoutedSTVState voters initialCandidates),
  self.elected ⊆ initialCandidates

field active_elected_disjoint:
∀ {Voter : Type u_1} {Candidate : Type u_2} [inst : DecidableEq Voter] [inst_1 : DecidableEq Candidate]
  {voters : Finset Voter} {initialCandidates : Finset Candidate}
  (self : GGRS26CombattingGerrymanderingRCV.BallotRoutedSTVState voters initialCandidates),
  Disjoint self.active self.elected

field weight_nonneg:
∀ {Voter : Type u_1} {Candidate : Type u_2} [inst : DecidableEq Voter] [inst_1 : DecidableEq Candidate]
  {voters : Finset Voter} {initialCandidates : Finset Candidate}
  (self : GGRS26CombattingGerrymanderingRCV.BallotRoutedSTVState voters initialCandidates),
  ∀ voter ∈ voters, 0 ≤ self.weight voter

```

GGRS26CombattingGerrymanderingRCV.BallotRoutedSTVState.initial

Declaration location: paper-local

Lean declaration body

```

type:
{Voter : Type u_1} →
{Candidate : Type u_2} →
  [inst : DecidableEq Voter] →
  [inst_1 : DecidableEq Candidate] →
  {voters : Finset Voter} →
  {initialCandidates : Finset Candidate} →
  (initialWeight : Voter → ℝ) →
  (∀ voter ∈ voters, 0 ≤ initialWeight voter) →
    GGRS26CombattingGerrymanderingRCV.BallotRoutedSTVState voters initialCandidates

value:
fun {Voter : Type u_1} {Candidate : Type u_2} [DecidableEq Voter] [DecidableEq Candidate] {voters : Finset Voter}
  {initialCandidates : Finset Candidate} (initialWeight : Voter → ℝ)
  (initialWeight_nonneg : ∀ voter ∈ voters, 0 ≤ initialWeight voter) =>
  { active := initialCandidates, elected := ∅, weight := initialWeight,
    active_subset_initial := fun {candidate : Candidate} {hcandidate : candidate ∈ initialCandidates} => hcandidate,
    elected_subset_initial := GGRS26CombattingGerrymanderingRCV.BallotRoutedSTVState.initial_proof_1,
    active_elected_disjoint := GGRS26CombattingGerrymanderingRCV.BallotRoutedSTVState.initial_proof_2,
    weight_nonneg := hinitialWeight_nonneg }

```

Direct retained dependencies

- [GGRS26CombattingGerrymanderingRCV.BallotRoutedSTVState](#)

GGRS26CombattingGerrymanderingRCV.BallotRoutedSTVTerminal

Declaration location: paper-local

Lean declaration body

```

type:
{Voter : Type u_1} →
{Candidate : Type u_2} →
  [inst : DecidableEq Voter] →
  [inst_1 : DecidableEq Candidate] →
  {voters : Finset Voter} →
  {initialCandidates : Finset Candidate} →
  ℕ → GGRS26CombattingGerrymanderingRCV.BallotRoutedSTVState voters initialCandidates → Prop

value:
fun {Voter : Type u_1} {Candidate : Type u_2} [DecidableEq Voter] [DecidableEq Candidate] {voters : Finset Voter}
  {initialCandidates : Finset Candidate} (seats : ℕ)
  (state : GGRS26CombattingGerrymanderingRCV.BallotRoutedSTVState voters initialCandidates) =>
  state.elected.card = seats v state.active.card = seats - state.elected.card

```

Direct retained dependencies

- [GGRS26CombattingGerrymanderingRCV.BallotRoutedSTVState](#)

GGRS26CombatingGerrymanderingRCV.PartyApprovalBallot

Declaration location: paper-local

Lean declaration body

```
type:
Type u_1 → Type u_1

value:
fun (Candidate : Type u_1) => AppliedModelingLib.SocialChoice.Voting.ApprovalBallot Candidate
```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Voting.ApprovalBallot](#)

GGRS26CombatingGerrymanderingRCV.ballotRoutedPartyFinalSeats

Declaration location: paper-local

Lean declaration body

```
type:
{Voter : Type u_1} →
{Candidate : Type u_2} →
[inst : DecidableEq Voter] →
[inst_1 : DecidableEq Candidate] →
{voters : Finset Voter} →
{initialCandidates : Finset Candidate} →
Finset Candidate → ℕ → GGRS26CombatingGerrymanderingRCV.BallotRoutedSTVState voters initialCandidates → ℕ

value:
fun {Voter : Type u_1} {Candidate : Type u_2} [DecidableEq Voter] [DecidableEq Candidate] {voters : Finset Voter}
{initialCandidates : Finset Candidate} (partyCandidates : Finset Candidate) (seats : ℕ)
(state : GGRS26CombatingGerrymanderingRCV.BallotRoutedSTVState voters initialCandidates) =>
(state.elected n partyCandidates).card +
if state.elected.card < seats then (state.active n partyCandidates).card else 0
```

Direct retained dependencies

- [GGRS26CombatingGerrymanderingRCV.BallotRoutedSTVState](#)

GGRS26CombatingGerrymanderingRCV.ballotRoutedSupport

Declaration location: paper-local

Lean declaration body

```
type:
{Voter : Type u_1} →
{Candidate : Type u_2} →
[DecidableEq Candidate] →
Finset Voter →
(Voter → AppliedModelingLib.SocialChoice.Voting.Ballot Candidate) → Finset Candidate → Candidate → Finset Voter

value:
fun {Voter : Type u_1} {Candidate : Type u_2} [DecidableEq Candidate] (voters : Finset Voter)
(ballots : Voter → AppliedModelingLib.SocialChoice.Voting.Ballot Candidate) (active : Finset Candidate)
(candidate : Candidate) =>
AppliedModelingLib.SocialChoice.Voting.Ballot.activeSupport voters ballots active candidate
```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Voting.Ballot](#)
- [AppliedModelingLib.SocialChoice.Voting.Ballot.activeSupport](#)

GGRS26CombatingGerrymanderingRCV.ballotRoutedTally

Declaration location: paper-local

Lean declaration body

```
type:
{Voter : Type u_1} →
{Candidate : Type u_2} →
[DecidableEq Candidate] →
Finset Voter →
(Voter → AppliedModelingLib.SocialChoice.Voting.Ballot Candidate) →
Finset Candidate → (Voter → ℝ) → Candidate → ℝ

value:
fun {Voter : Type u_1} {Candidate : Type u_2} [DecidableEq Candidate] (voters : Finset Voter)
(ballots : Voter → AppliedModelingLib.SocialChoice.Voting.Ballot Candidate) (active : Finset Candidate)
(weight : Voter → ℝ) (candidate : Candidate) =>
∑ voter ∈ GGRS26CombatingGerrymanderingRCV.ballotRoutedSupport voters ballots active candidate, weight voter
```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Voting.Ballot](#)
- [GGRS26CombatingGerrymanderingRCV.ballotRoutedSupport](#)

GGRS26CombatingGerrymanderingRCV.BallotRoutedSTVTransferPolicy

Declaration location: paper-local

Lean declaration body

```
field electUpdate:
{Voter : Type u_1} →
{Candidate : Type u_2} →
[inst : DecidableEq Voter] →
[inst_1 : DecidableEq Candidate] →
{voters : Finset Voter} →
{ballots : Voter → AppliedModelingLib.SocialChoice.Voting.Ballot Candidate} →
{quota : ℝ} →
GGRS26CombatingGerrymanderingRCV.BallotRoutedSTVTransferPolicy voters ballots quota →
Finset Candidate → Candidate → (Voter → ℝ) → (Voter → ℝ) → Prop

field eliminateUpdate:
{Voter : Type u_1} →
{Candidate : Type u_2} →
[inst : DecidableEq Voter] →
[inst_1 : DecidableEq Candidate] →
{voters : Finset Voter} →
{ballots : Voter → AppliedModelingLib.SocialChoice.Voting.Ballot Candidate} →
{quota : ℝ} →
GGRS26CombatingGerrymanderingRCV.BallotRoutedSTVTransferPolicy voters ballots quota →
Finset Candidate → Candidate → (Voter → ℝ) → (Voter → ℝ) → Prop

field elect_exists:
∀ {Voter : Type u_1} {Candidate : Type u_2} [inst : DecidableEq Voter] [inst_1 : DecidableEq Candidate]
{voters : Finset Voter} {ballots : Voter → AppliedModelingLib.SocialChoice.Voting.Ballot Candidate} {quota : ℝ}
(self : GGRS26CombatingGerrymanderingRCV.BallotRoutedSTVTransferPolicy voters ballots quota)
(active : Finset Candidate) (winner : Candidate) (beforeWeight : Voter → ℝ),
(∀ voter ∈ voters, 0 ≤ beforeWeight voter) →
winner ∈ active →
quota ≤ GGRS26CombatingGerrymanderingRCV.ballotRoutedTally voters ballots active beforeWeight winner →
∃ (afterWeight : Voter → ℝ), self.electUpdate active winner beforeWeight afterWeight

field eliminate_exists:
∀ {Voter : Type u_1} {Candidate : Type u_2} [inst : DecidableEq Voter] [inst_1 : DecidableEq Candidate]
{voters : Finset Voter} {ballots : Voter → AppliedModelingLib.SocialChoice.Voting.Ballot Candidate} {quota : ℝ}
(self : GGRS26CombatingGerrymanderingRCV.BallotRoutedSTVTransferPolicy voters ballots quota)
(active : Finset Candidate) (loser : Candidate) (beforeWeight : Voter → ℝ),
(∀ voter ∈ voters, 0 ≤ beforeWeight voter) →
loser ∈ active → ∃ (afterWeight : Voter → ℝ), self.eliminateUpdate active loser beforeWeight afterWeight

field elect_preserves_nonneg:
∀ {Voter : Type u_1} {Candidate : Type u_2} [inst : DecidableEq Voter] [inst_1 : DecidableEq Candidate]
{voters : Finset Voter} {ballots : Voter → AppliedModelingLib.SocialChoice.Voting.Ballot Candidate} {quota : ℝ}
(self : GGRS26CombatingGerrymanderingRCV.BallotRoutedSTVTransferPolicy voters ballots quota)
(active : Finset Candidate) (winner : Candidate) (beforeWeight afterWeight : Voter → ℝ),
(∀ voter ∈ voters, 0 ≤ beforeWeight voter) →
self.electUpdate active winner beforeWeight afterWeight → ∀ voter ∈ voters, 0 ≤ afterWeight voter

field elect_unchanged_off_support:
∀ {Voter : Type u_1} {Candidate : Type u_2} [inst : DecidableEq Voter] [inst_1 : DecidableEq Candidate]
{voters : Finset Voter} {ballots : Voter → AppliedModelingLib.SocialChoice.Voting.Ballot Candidate} {quota : ℝ}
(self : GGRS26CombatingGerrymanderingRCV.BallotRoutedSTVTransferPolicy voters ballots quota)
(active : Finset Candidate) (winner : Candidate) (beforeWeight afterWeight : Voter → ℝ),
(∀ voter ∈ voters, 0 ≤ beforeWeight voter) →
self.electUpdate active winner beforeWeight afterWeight →
∀ voter ∈ voters,
voter ∉ GGRS26CombatingGerrymanderingRCV.ballotRoutedSupport voters ballots active winner →
afterWeight voter = beforeWeight voter

field elect_support_mass_drop_exactly_quota:
∀ {Voter : Type u_1} {Candidate : Type u_2} [inst : DecidableEq Voter] [inst_1 : DecidableEq Candidate]
{voters : Finset Voter} {ballots : Voter → AppliedModelingLib.SocialChoice.Voting.Ballot Candidate} {quota : ℝ}
```

```

(self : GGRS26CombattingGerrymanderingRCV.BallotRoutedSTVTransferPolicy voters ballots quota)
(active : Finset Candidate) (winner : Candidate) (beforeWeight afterWeight : Voter → ℝ),
(∀ voter ∈ voters, 0 ≤ beforeWeight voter) →
self.electUpdate active winner beforeWeight afterWeight →
  Σ voter ∈ GGRS26CombattingGerrymanderingRCV.BallotRoutedSTVTransferPolicy voters ballots active winner,
  Σ voter ∈ GGRS26CombattingGerrymanderingRCV.BallotRoutedSupport voters ballots active winner,
  beforeWeight voter -
  quota

```

field eliminate_weight_unchanged:

```

∀ {Voter : Type u_1} {Candidate : Type u_2} [inst : DecidableEq Voter] [inst_1 : DecidableEq Candidate]
{voters : Finset Voter} {ballots : Voter → AppliedModelingLib.SocialChoice.Voting.Ballot Candidate} {quota : ℝ}
(self : GGRS26CombattingGerrymanderingRCV.BallotRoutedSTVTransferPolicy voters ballots quota)
(active : Finset Candidate) (loser : Candidate) (beforeWeight afterWeight : Voter → ℝ),
(∀ voter ∈ voters, 0 ≤ beforeWeight voter) →
self.eliminateUpdate active loser beforeWeight afterWeight →
  ∀ voter ∈ voters, afterWeight voter = beforeWeight voter

```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Voting.Ballot](#)
- [GGRS26CombattingGerrymanderingRCV.ballotRoutedSupport](#)
- [GGRS26CombattingGerrymanderingRCV.ballotRoutedTally](#)

GGRS26CombattingGerrymanderingRCV.BallotRoutedDFavoringSTVTransition

Declaration location: paper-local

Lean declaration body

```

type:
{Voter : Type u_1} →
{Candidate : Type u_2} →
[inst : DecidableEq Voter] →
[inst_1 : DecidableEq Candidate] →
{voters : Finset Voter} →
{initialCandidates : Finset Candidate} →
{favoredParty : Finset Candidate} →
(ballots : Voter → AppliedModelingLib.SocialChoice.Voting.Ballot Candidate) →
(quota : ℝ) →
  GGRS26CombattingGerrymanderingRCV.BallotRoutedSTVTransferPolicy voters ballots quota →
  ℕ →
  GGRS26CombattingGerrymanderingRCV.BallotRoutedSTVState voters initialCandidates →
  GGRS26CombattingGerrymanderingRCV.BallotRoutedSTVState voters initialCandidates → Prop

constructor GGRS26CombattingGerrymanderingRCV.BallotRoutedDFavoringSTVTransition.elect:
∀ {Voter : Type u_1} {Candidate : Type u_2} [inst : DecidableEq Voter] [inst_1 : DecidableEq Candidate]
{voters : Finset Voter} {initialCandidates : Finset Candidate}
{ballots : Voter → AppliedModelingLib.SocialChoice.Voting.Ballot Candidate} {quota : ℝ}
{policy : GGRS26CombattingGerrymanderingRCV.BallotRoutedSTVTransferPolicy voters ballots quota} {seats : ℕ}
{before after : GGRS26CombattingGerrymanderingRCV.BallotRoutedSTVState voters initialCandidates} (winner : Candidate),
¬GGRS26CombattingGerrymanderingRCV.BallotRoutedSTVTerminal seats before →
winner ∈ before.active →
before.elected.card < seats →
quota ≤ GGRS26CombattingGerrymanderingRCV.ballotRoutedTally voters ballots before.active before.weight winner →
(winner ∉ favoredParty →
  ∀ candidate ∈ before.active,
    GGRS26CombattingGerrymanderingRCV.ballotRoutedTally voters ballots before.active before.weight
    candidate =
    GGRS26CombattingGerrymanderingRCV.ballotRoutedTally voters ballots before.active before.weight
    winner →
    candidate ∉ favoredParty) →
after.active = before.active.erase winner →
after.elected = insert winner before.elected →
policy.electUpdate before.active winner before.weight after.weight →
  GGRS26CombattingGerrymanderingRCV.BallotRoutedDFavoringSTVTransition ballots quota policy seats before
  after

constructor GGRS26CombattingGerrymanderingRCV.BallotRoutedDFavoringSTVTransition.eliminate:
∀ {Voter : Type u_1} {Candidate : Type u_2} [inst : DecidableEq Voter] [inst_1 : DecidableEq Candidate]
{voters : Finset Voter} {initialCandidates : Finset Candidate}
{ballots : Voter → AppliedModelingLib.SocialChoice.Voting.Ballot Candidate} {quota : ℝ}
{policy : GGRS26CombattingGerrymanderingRCV.BallotRoutedSTVTransferPolicy voters ballots quota} {seats : ℕ}
{before after : GGRS26CombattingGerrymanderingRCV.BallotRoutedSTVState voters initialCandidates} (loser : Candidate),
¬GGRS26CombattingGerrymanderingRCV.BallotRoutedSTVTerminal seats before →
loser ∈ before.active →
(∀ candidate ∈ before.active,
  GGRS26CombattingGerrymanderingRCV.ballotRoutedTally voters ballots before.active before.weight candidate <
  quota) →
(∀ candidate ∈ before.active,
  GGRS26CombattingGerrymanderingRCV.ballotRoutedTally voters ballots before.active before.weight loser ≤
  GGRS26CombattingGerrymanderingRCV.ballotRoutedTally voters ballots before.active before.weight
  candidate) →
(loser ∈ favoredParty →
  ∀ candidate ∈ before.active,
    GGRS26CombattingGerrymanderingRCV.ballotRoutedTally voters ballots before.active before.weight
    candidate =
    GGRS26CombattingGerrymanderingRCV.ballotRoutedTally voters ballots before.active before.weight

```

```

    loser →
      candidate ∈ favoredParty) →
after.active = before.active.erase loser →
after.elected = before.elected →
policy.eliminateUpdate before.active loser before.weight after.weight →
GGRS26CombatingGerrymanderingRCV.BallotRoutedDFavoringSTVTransition ballots quota policy seats before
after

```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Voting.Ballot](#)
- [GGRS26CombatingGerrymanderingRCV.BallotRoutedSTVState](#)
- [GGRS26CombatingGerrymanderingRCV.BallotRoutedSTVTerminal](#)
- [GGRS26CombatingGerrymanderingRCV.BallotRoutedSTVTransferPolicy](#)
- [GGRS26CombatingGerrymanderingRCV.ballotRoutedTally](#)

GGRS26CombatingGerrymanderingRCV.BallotRoutedDFavoringSTVRun

Declaration location: paper-local

Lean declaration body

```

type:
{Voter : Type u_1} →
{Candidate : Type u_2} →
[inst : DecidableEq Voter] →
[inst_1 : DecidableEq Candidate] →
{voters : Finset Voter} →
{initialCandidates : Finset Candidate} →
{favoredParty : Finset Candidate} →
(ballots : Voter → AppliedModelingLib.SocialChoice.Voting.Ballot Candidate) →
(quota : ℝ) →
  GGRS26CombatingGerrymanderingRCV.BallotRoutedSTVTransferPolicy voters ballots quota →
  ℕ →
    GGRS26CombatingGerrymanderingRCV.BallotRoutedSTVState voters initialCandidates →
    GGRS26CombatingGerrymanderingRCV.BallotRoutedSTVState voters initialCandidates → Prop

value:
fun {Voter : Type u_1} {Candidate : Type u_2} [DecidableEq Voter] [DecidableEq Candidate] {voters : Finset Voter}
  {initialCandidates favoredParty : Finset Candidate}
  (ballots : Voter → AppliedModelingLib.SocialChoice.Voting.Ballot Candidate) (quota : ℝ)
  (policy : GGRS26CombatingGerrymanderingRCV.BallotRoutedSTVTransferPolicy voters ballots quota) (seats : ℕ)
  (initial terminal : GGRS26CombatingGerrymanderingRCV.BallotRoutedSTVState voters initialCandidates) =>
Relation.ReflTransGen
  (GGRS26CombatingGerrymanderingRCV.BallotRoutedDFavoringSTVTransition ballots quota policy seats) initial terminal

```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Voting.Ballot](#)
- [GGRS26CombatingGerrymanderingRCV.BallotRoutedDFavoringSTVTransition](#)
- [GGRS26CombatingGerrymanderingRCV.BallotRoutedSTVState](#)
- [GGRS26CombatingGerrymanderingRCV.BallotRoutedSTVTransferPolicy](#)

GGRS26CombatingGerrymanderingRCV.paper_complete_ranked_ballots

Declaration location: paper-local

Lean declaration body

```

type:
{Voter : Type u_1} →
{Candidate : Type u_2} →
[DecidableEq Candidate] →
  Finset Voter → (Voter → AppliedModelingLib.SocialChoice.Voting.Ballot Candidate) → Finset Candidate → Prop

value:
fun {Voter : Type u_1} {Candidate : Type u_2} [DecidableEq Candidate] (voters : Finset Voter)
  (ballots : Voter → AppliedModelingLib.SocialChoice.Voting.Ballot Candidate) (allCandidates : Finset Candidate) =>
  ∀ voter ∈ voters,
    (ballots voter).Valid ∧ ∀ (candidate : Candidate), candidate ∈ ballots voter ↔ candidate ∈ allCandidates

```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Voting.Ballot](#)
- [AppliedModelingLib.SocialChoice.Voting.Ballot.Valid](#)

GGRS26CombatingGerrymanderingRCV.paper_complete_party_ranking_ballots

Declaration location: paper-local

Lean declaration body

```
type:
  {Voter : Type u_1} →
  {Candidate : Type u_2} →
  [DecidableEq Candidate] →
  Finset Voter →
  (Voter → AppliedModelingLib.SocialChoice.Voting.Ballot Candidate) → Finset Candidate → Finset Candidate → Prop

value:
  fun {Voter : Type u_1} {Candidate : Type u_2} [DecidableEq Candidate] (voters : Finset Voter)
    (ballots : Voter → AppliedModelingLib.SocialChoice.Voting.Ballot Candidate)
    (partyCandidates allCandidates : Finset Candidate) =>
  GGRS26CombatingGerrymanderingRCV.paper_complete_ranked_ballots voters ballots allCandidates ∧
  ∀ voter ∈ voters,
  ∀ (active : Finset Candidate),
  (∃ same ∈ partyCandidates, same ∈ active) →
  ∃ same ∈ partyCandidates, (ballots voter).nextActive active = some same
```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Voting.Ballot](#)
- [AppliedModelingLib.SocialChoice.Voting.Ballot.nextActive](#)
- [GGRS26CombatingGerrymanderingRCV.paper_complete_ranked_ballots](#)

GGRS26CombatingGerrymanderingRCV.paper_pav_thiele_weights

Declaration location: paper-local

Lean declaration body

```
type:
  GGRS26CombatingGerrymanderingRCV.PaperThieleWeights

value:
  { marginalWeight := AppliedModelingLib.SocialChoice.Voting.pavWeight,
    marginal_nonincreasing := GGRS26CombatingGerrymanderingRCV.paper_pav_thiele_weights._proof_1,
    voterScore := AppliedModelingLib.SocialChoice.Voting.pavHarmonicSum,
    voterScore_zero := GGRS26CombatingGerrymanderingRCV.paper_pav_thiele_weights._proof_2,
    voterScore_succ := GGRS26CombatingGerrymanderingRCV.paper_pav_thiele_weights._proof_3 }
```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Voting.pavHarmonicSum](#)
- [AppliedModelingLib.SocialChoice.Voting.pavWeight](#)
- [GGRS26CombatingGerrymanderingRCV.PaperThieleWeights](#)

GGRS26CombatingGerrymanderingRCV.paper_proposition1_ballot_routed_stv_initial_state

Declaration location: paper-local

Lean declaration body

```
type:
  {Voter : Type u_1} →
  {Candidate : Type u_2} →
  [inst : DecidableEq Voter] →
  [inst_1 : DecidableEq Candidate] →
  (allVoters : Finset Voter) →
  (initialActive : Finset Candidate) →
  GGRS26CombatingGerrymanderingRCV.BallotRoutedSTVState allVoters initialActive

value:
  fun {Voter : Type u_1} {Candidate : Type u_2} [DecidableEq Voter] [DecidableEq Candidate] (allVoters : Finset Voter)
    (initialActive : Finset Candidate) =>
  GGRS26CombatingGerrymanderingRCV.BallotRoutedSTVState.initial (fun (x : Voter) => 1)
  (GGRS26CombatingGerrymanderingRCV.paper_proposition1_ballot_routed_stv_initial_state._proof_1 allVoters)
```

Direct retained dependencies

- [GGRS26CombatingGerrymanderingRCV.BallotRoutedSTVState](#)
- [GGRS26CombatingGerrymanderingRCV.BallotRoutedSTVState.initial](#)

GGRS26CombatingGerrymanderingRCV.paper_thiele_party_seat_score

Declaration location: paper-local

Lean declaration body

```
type:
GGRS26CombatingGerrymanderingRCV.PaperThieleWeights → ℝ → ℕ → ℕ → ℝ

value:
fun (weights : GGRS26CombatingGerrymanderingRCV.PaperThieleWeights) (partyShare : ℝ) (seats seatCount : ℕ) =>
  partyShare * weights.voterScore seatCount + (1 - partyShare) * weights.voterScore (seats - seatCount)
```

Direct retained dependencies

- [GGRS26CombatingGerrymanderingRCV.PaperThieleWeights](#)

GGRS26CombatingGerrymanderingRCV.paper_pav_min_argmax

Declaration location: paper-local

Lean declaration body

```
type:
ℕ → ℝ → ℕ → Prop

value:
fun (seatCount : ℕ) (partyShare : ℝ) (seats : ℕ) =>
  GGRS26CombatingGerrymanderingRCV.paper_thiele_party_min_argmax
    GGRS26CombatingGerrymanderingRCV.paper_pav_thiele_weights seatCount partyShare seats
```

Direct retained dependencies

- [GGRS26CombatingGerrymanderingRCV.paper_pav_thiele_weights](#)
- [GGRS26CombatingGerrymanderingRCV.paper_thiele_party_min_argmax](#)

GGRS26CombatingGerrymanderingRCV.pavSeatIntegerInterval

Declaration location: paper-local

Lean declaration body

```
type:
ℤ → ℝ → ℕ → Prop

value:
fun (seatCount : ℤ) (partyShare : ℝ) (seats : ℕ) =>
  partyShare * ↑(seats + 1) - 1 ≤ ↑seatCount ∧ ↑seatCount < partyShare * ↑(seats + 1)
```

GGRS26CombatingGerrymanderingRCV.paper_pav_integer_interval

Declaration location: paper-local

Lean declaration body

```
type:
ℤ → ℝ → ℕ → Prop

value:
fun (seatCount : ℤ) (partyShare : ℝ) (seats : ℕ) =>
  GGRS26CombatingGerrymanderingRCV.pavSeatIntegerInterval seatCount partyShare seats
```

Direct retained dependencies

- [GGRS26CombatingGerrymanderingRCV.pavSeatIntegerInterval](#)

GGRS26CombatingGerrymanderingRCV.seatShareRounded

Declaration location: paper-local

Lean declaration body

```
type:
 $\mathbb{N} \rightarrow \mathbb{R} \rightarrow \mathbb{N} \rightarrow \text{Prop}$ 

value:
fun (seatCount :  $\mathbb{N}$ ) (partyShare :  $\mathbb{R}$ ) (seats :  $\mathbb{N}$ ) =>
  AppliedModelingLib.SocialChoice.Voting.roundedSeatShare seatCount partyShare seats
```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Voting.roundedSeatShare](#)

GGRS26CombattingGerrymanderingRCV.paper_seat_share_rounded

Declaration location: paper-local

Lean declaration body

```
type:
 $\mathbb{N} \rightarrow \mathbb{R} \rightarrow \mathbb{N} \rightarrow \text{Prop}$ 

value:
fun (seatCount :  $\mathbb{N}$ ) (partyShare :  $\mathbb{R}$ ) (seats :  $\mathbb{N}$ ) =>
  GGRS26CombattingGerrymanderingRCV.seatShareRounded seatCount partyShare seats
```

Direct retained dependencies

- [GGRS26CombattingGerrymanderingRCV.seatShareRounded](#)

Paper-specific semantic prerequisites

GGRS26CombatingGerrymanderingRCV.PaperSourceSTVState

Paper-local declaration:

GGRS26CombatingGerrymanderingRCV.PaperSourceSTVState

Source locator: cited publication, lines 394-408

Verbatim paper-source connection

In Single Transferable Vote (STV), a type of ranked choice voting for multiple winners, each voter instead submits a ranking over candidates (here, we assume the rankings are complete, not partial). Consider a district with N_k seats and V_k voters; the set of winners is created iteratively, as follows. The number of first-place votes for each candidate is counted. Any candidate with a number of votes at least the “Droop” threshold $Q = \lfloor NV_k k+1 \rfloor + 1$ is selected as a winner, and their surplus votes (number of votes minus Q) are transferred to each voter’s next preference. If no candidate has enough votes, then the candidate with the least number of votes (with tie-breaking) is eliminated, and their votes are transferred. The process continues until N_k candidates have been selected, or the number of remaining candidates equals the number of remaining seats. STV is commonly used in practice in multi-winner elections.

Details can differ in how such votes are transferred, but our results in Section 3 do not depend on the transfer rule; in our simulations in Section 4 and Appendix B, we use the “fractional” STV rule, in which each voter keeps a fractional number of votes equal to their share of the winning candidate’s surplus votes (we describe this formally in Section 4). For consistency, for all rules we assume that ties are broken in favor of candidates from party D, but randomly within each party.⁵

Lean-elaborated paper signature

field active:

{Voter : Type u_1} →
{Candidate : Type u_2} → GGRS26CombatingGerrymanderingRCV.PaperSourceSTVState Voter Candidate → Finset Candidate

field elected:

{Voter : Type u_1} →
{Candidate : Type u_2} → GGRS26CombatingGerrymanderingRCV.PaperSourceSTVState Voter Candidate → Finset Candidate

field weight:

{Voter : Type u_1} →
{Candidate : Type u_2} → GGRS26CombatingGerrymanderingRCV.PaperSourceSTVState Voter Candidate → Voter → ℝ

Recorded source-to-declaration screening Verdict: matches

Reason: Exposes the active and elected candidate sets and current ballot weights that comprise the source STV count state; its execution predicate supplies the associated operational constraints.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GGRS26CombatingGerrymanderingRCV.PaperThieleWeights

Paper-local declaration:

GGRS26CombatingGerrymanderingRCV.PaperThieleWeights

Source locator: cited publication, lines 377-393

Verbatim paper-source connection

A Thiele rule is characterized by a function λ . Each voter v in district k provides a set S_v of the N_k candidates they like most. The set of winners is determined as follows. Consider a potential set of winners C . The amount of points that voter v contributes to C is $i=1 \dots \lambda(i)$, and $|S_v \cap C|$

the set C with the most points across voters (after tie-breaking) is selected. The (non-increasing) function λ establishes that votes have diminishing returns; the i th approved candidate in the set is worth $\lambda(i)$. For example, winner-take-all approval voting is a Thiele rule with $\lambda(i) = 1$: for each voter, the set C receives a number of points equal to the number of candidates in the set that the voter likes, and so the N_k candidates with the most votes win. We further consider Proportional Approval Voting (PAV), a Thiele rule with $\lambda_{PAV}(i) = 1/i$; and Thiele Squared, with $\lambda_{TS}(i) = 1/i^2$. PAV is well-studied in the theoretical social choice literature and comes with optimality guarantees (in terms of proportionality) (Skowron 2021). Thiele Squared induces even sharper diminishing returns than PAV in a voter having more of their preferred candidates winning, and thus prioritizes more voters having at least one approved candidate (and 50/50 outcomes between two parties, regardless of underlying vote shares). More generally, Thiele rules both contain common voting rules and parameterize a rule's tendency to favor proportional or balanced outcomes.

Lean-elaborated paper signature

field marginalWeight:

GGRS26CombatingGerrymanderingRCV.PaperThieleWeights $\rightarrow \mathbb{N} \rightarrow \mathbb{R}$

field marginal_nonincreasing:

$\forall$ (self : GGRS26CombatingGerrymanderingRCV.PaperThieleWeights) (i j : $\mathbb{N}$),
 $1 \leq i \rightarrow i \leq j \rightarrow \text{self.marginalWeight } j \leq \text{self.marginalWeight } i$

field voterScore:

GGRS26CombatingGerrymanderingRCV.PaperThieleWeights $\rightarrow \mathbb{N} \rightarrow \mathbb{R}$

field voterScore_zero:

$\forall$ (self : GGRS26CombatingGerrymanderingRCV.PaperThieleWeights), self.voterScore 0 = 0

field voterScore_succ:

$\forall$ (self : GGRS26CombatingGerrymanderingRCV.PaperThieleWeights) (n : $\mathbb{N}$),
self.voterScore (n + 1) = self.voterScore n + self.marginalWeight (n + 1)

Recorded source-to-declaration screening Verdict: matches

Reason: Its positive-index nonincreasing marginals, zero cumulative score, and successor recurrence encode the source Thiele sum from $i=1$ through n , including diminishing returns.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GGRS26CombatingGerrymanderingRCV.paper_candidate_ranked_before

Paper-local declaration:

GGRS26CombatingGerrymanderingRCV.paper_candidate_ranked_before

Source locator: cited publication, lines 415-426

Verbatim paper-source connection

Voter assumptions. Next, we require assumptions on how voters rank or approve candidates. For each Thiele rule, we assume that in each district k there are exactly N_k candidates from each party, and each voter simply approves all the candidates in their party.⁶ For STV, we assume that in each district k there are at least N_k candidates from each party.⁷ For our primarily empirical analysis, we further assume that parties are solid coalitions (Dummett 1984): that each voter ranks all candidates of their party over each candidate of the other party, though intra-party rankings may vary. These assumptions reflect that party membership is a strong predictor of the party of the candidate for which a voter votes. Such an assumption, using historical party vote shares to predict vote shares in future elections, is used throughout the redistricting literature for SMDs; our additional assumption is that this behavior extends to partisan voting in multi-member elections. We relax this assumption in Section 4 and study intra-party effects, where voters may disagree on within-party rankings, in Appendix B.

Lean-expanded paper semantic target

type:
{Candidate : Type u_1} → Candidate → Candidate → AppliedModelingLib.SocialChoice.Voting.Ballot Candidate → Prop

value:
fun {Candidate : Type u_1} (higher lower : Candidate)
 (ballot : AppliedModelingLib.SocialChoice.Voting.Ballot Candidate) =>
 ∃ (earlier : AppliedModelingLib.SocialChoice.Voting.Ballot Candidate) (middle : List Candidate) (suffix :
 List Candidate), ballot = earlier ++ higher :: middle ++ lower :: suffix

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Voting.Ballot](#)

Recorded source-to-declaration screening Verdict: matches

Reason: Its list decomposition requires the higher candidate to occur strictly before the lower candidate, which is the exact ranking relation needed for source solid coalitions.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GGRS26CombatingGerrymanderingRCV.paper_complete_solid_coalition_rankings

Paper-local declaration:

GGRS26CombatingGerrymanderingRCV.paper_complete_solid_coalition_rankings

Source locator: cited publication, lines 415-426

Verbatim paper-source connection

Voter assumptions. Next, we require assumptions on how voters rank or approve candidates. For each Thiele rule, we assume that in each district k there are exactly N_k candidates from each party, and each voter simply approves all the candidates in their party.⁶ For STV, we assume that in each district k there are at least N_k candidates from each party.⁷ For our primarily empirical analysis, we further assume that parties are solid coalitions (Dummett 1984): that each voter ranks all candidates of their party over each candidate of the other party, though intra-party rankings may vary. These assumptions reflect that party membership is a strong predictor of the party of the candidate for which a voter votes. Such an assumption, using historical party vote shares to predict vote shares in future elections, is used throughout the redistricting literature for SMDs; our additional assumption is that this behavior extends to partisan voting in multi-member elections. We relax this assumption in Section 4 and study intra-party effects, where voters may disagree on within-party rankings, in Appendix B.

Lean-expanded paper semantic target

```
type:
{Voter : Type u_1} →
{Candidate : Type u_2} →
  Finset Voter →
    (Voter → AppliedModelingLib.SocialChoice.Voting.Ballot Candidate) →
      Finset Candidate → Finset Candidate → Finset Candidate → Prop

value:
fun {Voter : Type u_1} {Candidate : Type u_2} (voters : Finset Voter)
  (ballots : Voter → AppliedModelingLib.SocialChoice.Voting.Ballot Candidate)
  (partyCandidates otherPartyCandidates allCandidates : Finset Candidate) =>
  Disjoint partyCandidates otherPartyCandidates ∧
  allCandidates = partyCandidates ∪ otherPartyCandidates ∧
  ∀ voter ∈ voters,
    (ballots voter).Valid ∧
    (∀ (candidate : Candidate), candidate ∈ ballots voter ↔ candidate ∈ allCandidates) ∧
    ∀ same ∈ partyCandidates,
      ∀ other ∈ otherPartyCandidates,
        GGRS26CombatingGerrymanderingRCV.paper_candidate_ranked_before same other (ballots voter)
```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Voting.Ballot](#)
- [AppliedModelingLib.SocialChoice.Voting.Ballot.Valid](#)
- [GGRS26CombatingGerrymanderingRCV.paper_candidate_ranked_before](#)

Recorded source-to-declaration screening Verdict: matches

Reason: Uses disjoint complete two-party rosters and requires every valid full ballot to rank each own-party candidate above every other-party candidate, while leaving within-party ordering unrestricted as in the source.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GGRS26CombatingGerrymanderingRCV.paper_first_active_candidate

Paper-local declaration:

GGRS26CombatingGerrymanderingRCV.paper_first_active_candidate

Source locator: cited publication, lines 394-408

Verbatim paper-source connection

In Single Transferable Vote (STV), a type of ranked choice voting for multiple winners, each voter instead submits a ranking over candidates (here, we assume the rankings are complete, not partial). Consider a district with N_k seats and V_k voters; the set of winners is created iteratively, as follows. The number of first-place votes for each candidate is counted. Any candidate with a number of votes at least the “Droop” threshold $Q = \lfloor NV_k / k + 1 \rfloor + 1$ is selected as a winner, and their surplus votes (number of votes minus Q) are transferred to each voter’s next preference. If no candidate has enough votes, then the candidate with the least number of votes (with tie-breaking) is eliminated, and their votes are transferred. The process continues until N_k candidates have been selected, or the number of remaining candidates equals the number of remaining seats. STV is commonly used in practice in multi-winner elections.

Details can differ in how such votes are transferred, but our results in Section 3 do not depend on the transfer rule; in our simulations in Section 4 and Appendix B, we use the “fractional” STV rule, in which each voter keeps a fractional number of votes equal to their share of the winning candidate’s surplus votes (we describe this formally in Section 4). For consistency, for all rules we assume that ties are broken in favor of candidates from party D, but randomly within each party.⁵

Lean-expanded paper semantic target

type:

{Candidate : Type u_1} → AppliedModelingLib.SocialChoice.Voting.Ballot Candidate → Finset Candidate → Candidate → Prop

value:

```
fun {Candidate : Type u_1} (ballot : AppliedModelingLib.SocialChoice.Voting.Ballot Candidate)
  (active : Finset Candidate) (candidate : Candidate) =>
  candidate ∈ active ∧
  ∃ (preceding : AppliedModelingLib.SocialChoice.Voting.Ballot Candidate) (suffix : List Candidate),
    ballot = preceding ++ candidate :: suffix ∧ ∀ earlier ∈ preceding, earlier ∉ active
```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Voting.Ballot](#)

Recorded source-to-declaration screening Verdict: matches

Reason: Requires an active candidate with every earlier listed candidate inactive, exactly the source rule’s first currently active preference.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GGRS26CombatingGerrymanderingRCV.paper_thiele_committee_score

Paper-local declaration:

GGRS26CombatingGerrymanderingRCV.paper_thiele_committee_score

Source locator: cited publication, lines 377-393

Verbatim paper-source connection

A Thiele rule is characterized by a function λ . Each voter v in district k provides a set S_v of the N_k candidates they like most. The set of winners is determined as follows. Consider a potential set of winners C . The amount of points that voter v contributes to C is $\sum_{i=1}^{|S_v \cap C|} \lambda(i)$, and

the set C with the most points across voters (after tie-breaking) is selected. The (non-increasing) function λ establishes that votes have diminishing returns; the i th approved candidate in the set is worth $\lambda(i)$. For example, winner-take-all approval voting is a Thiele rule with $\lambda(i) = 1$: for each voter, the set C receives a number of points equal to the number of candidates in the set that the voter likes, and so the N_k candidates with the most votes win. We further consider Proportional Approval Voting (PAV), a Thiele rule with $\lambda_{\text{PAV}}(i) = 1/i$; and Thiele Squared, with $\lambda_{\text{TS}}(i) = 1/i^2$. PAV is well-studied in the theoretical social choice literature and comes with optimality guarantees (in terms of proportionality) (Skowron 2021). Thiele Squared induces even sharper diminishing returns than PAV in a voter having more of their preferred candidates winning, and thus prioritizes more voters having at least one approved candidate (and 50/50 outcomes between two parties, regardless of underlying vote shares). More generally, Thiele rules both contain common voting rules and parameterize a rule's tendency to favor proportional or balanced outcomes.

Lean-expanded paper semantic target

type:

```
{Candidate : Type u_1} →  
  GGRS26CombatingGerrymanderingRCV.PaperThieleWeights →  
  Finset Candidate → List (GGRS26CombatingGerrymanderingRCV.PartyApprovalBallot Candidate) → ℝ
```

value:

```
fun {Candidate : Type u_1} (weights : GGRS26CombatingGerrymanderingRCV.PaperThieleWeights)  
  (committee : Finset Candidate) (profile : List (GGRS26CombatingGerrymanderingRCV.PartyApprovalBallot Candidate)) =>  
  AppliedModelingLib.SocialChoice.Voting.thieleScore weights.voterScore committee profile
```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Voting.thieleScore](#)
- [GGRS26CombatingGerrymanderingRCV.PaperThieleWeights](#)
- [GGRS26CombatingGerrymanderingRCV.PartyApprovalBallot](#)

Recorded source-to-declaration screening Verdict: matches

Reason: Maps each approval set to the cumulative score at its intersection count with the committee and sums across voters, exactly the source Thiele committee-score formula.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GGRS26CombatingGerrymanderingRCV.paper_thiele_fixed_size_selected_committee

Paper-local declaration:

GGRS26CombatingGerrymanderingRCV.paper_thiele_fixed_size_selected_committee

Source locator: cited publication, lines 377-393

Verbatim paper-source connection

A Thiele rule is characterized by a function λ . Each voter v in district k provides a set S_v of the N_k candidates they like most. The set of winners is determined as follows. Consider a potential set of winners C . The amount of points that voter v contributes to C is $\sum_{i=1}^{|S_v \cap C|} \lambda(i)$, and

the set C with the most points across voters (after tie-breaking) is selected. The (non-increasing) function λ establishes that votes have diminishing returns; the i th approved candidate in the set is worth $\lambda(i)$. For example, winner-take-all approval voting is a Thiele rule with $\lambda(i) = 1$: for each voter, the set C receives a number of points equal to the number of candidates in the set that the voter likes, and so the N_k candidates with the most votes win. We further consider Proportional Approval Voting (PAV), a Thiele rule with $\lambda_{PAV}(i) = 1/i$; and Thiele Squared, with $\lambda_{TS}(i) = 1/i^2$. PAV is well-studied in the theoretical social choice literature and comes with optimality guarantees (in terms of proportionality) (Skowron 2021). Thiele Squared induces even sharper diminishing returns than PAV in a voter having more of their preferred candidates winning, and thus prioritizes more voters having at least one approved candidate (and 50/50 outcomes between two parties, regardless of underlying vote shares). More generally, Thiele rules both contain common voting rules and parameterize a rule's tendency to favor proportional or balanced outcomes.

Lean-expanded paper semantic target

type:

```
{Candidate : Type u_1} →  
  GGRS26CombatingGerrymanderingRCV.PaperThieleWeights →  
    (Finset Candidate → Finset Candidate → Prop) →  
      Finset Candidate →  
        Finset Candidate → List (GGRS26CombatingGerrymanderingRCV.PartyApprovalBallot Candidate) → ℕ → Prop
```

value:

```
fun {Candidate : Type u_1} (weights : GGRS26CombatingGerrymanderingRCV.PaperThieleWeights)  
  (tiePreferred : Finset Candidate → Finset Candidate → Prop) (candidates committee : Finset Candidate)  
  (profile : List (GGRS26CombatingGerrymanderingRCV.PartyApprovalBallot Candidate)) (seats : ℕ) =>  
  committee ⊆ candidates ∧  
  committee.card = seats ∧  
  (∀ other ⊆ candidates,  
    other.card = seats →  
      GGRS26CombatingGerrymanderingRCV.paper_thiele_committee_score weights other profile ≤  
      GGRS26CombatingGerrymanderingRCV.paper_thiele_committee_score weights committee profile) ∧  
  ∀ other ⊆ candidates,  
    other.card = seats →  
      GGRS26CombatingGerrymanderingRCV.paper_thiele_committee_score weights other profile =  
      GGRS26CombatingGerrymanderingRCV.paper_thiele_committee_score weights committee profile →  
      tiePreferred committee other
```

Direct retained dependencies

- [GGRS26CombatingGerrymanderingRCV.PaperThieleWeights](#)
- [GGRS26CombatingGerrymanderingRCV.PartyApprovalBallot](#)
- [GGRS26CombatingGerrymanderingRCV.paper_thiele_committee_score](#)

Recorded source-to-declaration screening Verdict: matches

Reason: Requires a subset of the candidate roster of the fixed district size with globally maximal Thiele score, resolving equal maximizers by the supplied tie preference as the source specifies.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GGRS26CombatingGerrymanderingRCV.paper_pav_fixed_size_selected_committee

Paper-local declaration:

GGRS26CombatingGerrymanderingRCV.paper_pav_fixed_size_selected_committee

Source locator: cited publication, lines 377-393

Verbatim paper-source connection

A Thiele rule is characterized by a function λ . Each voter v in district k provides a set S_v of the N_k candidates they like most. The set of winners is determined as follows. Consider a potential set of winners C . The amount of points that voter v contributes to C is $\sum_{i=1}^{|S_v \cap C|} \lambda(i)$, and

the set C with the most points across voters (after tie-breaking) is selected. The (non-increasing) function λ establishes that votes have diminishing returns; the i th approved candidate in the set is worth $\lambda(i)$. For example, winner-take-all approval voting is a Thiele rule with $\lambda(i) = 1$: for each voter, the set C receives a number of points equal to the number of candidates in the set that the voter likes, and so the N_k candidates with the most votes win. We further consider Proportional Approval Voting (PAV), a Thiele rule with $\lambda_{PAV}(i) = 1/i$; and Thiele Squared, with $\lambda_{TS}(i) = 1/i^2$. PAV is well-studied in the theoretical social choice literature and comes with optimality guarantees (in terms of proportionality) (Skowron 2021). Thiele Squared induces even sharper diminishing returns than PAV in a voter having more of their preferred candidates winning, and thus prioritizes more voters having at least one approved candidate (and 50/50 outcomes between two parties, regardless of underlying vote shares). More generally, Thiele rules both contain common voting rules and parameterize a rule's tendency to favor proportional or balanced outcomes.

Lean-expanded paper semantic target

type:

```
{Candidate : Type u_1} →  
(Finset Candidate → Finset Candidate → Prop) →  
  Finset Candidate →  
  Finset Candidate → List (GGRS26CombatingGerrymanderingRCV.PartyApprovalBallot Candidate) → ℕ → Prop
```

value:

```
fun {Candidate : Type u_1} (tiePreferred : Finset Candidate → Finset Candidate → Prop)  
  (candidates committee : Finset Candidate)  
  (profile : List (GGRS26CombatingGerrymanderingRCV.PartyApprovalBallot Candidate)) (seats : ℕ) =>  
  GGRS26CombatingGerrymanderingRCV.paper_thiele_fixed_size_selected_committee  
  GGRS26CombatingGerrymanderingRCV.paper_pav_thiele_weights tiePreferred candidates committee profile seats
```

Direct retained dependencies

- [GGRS26CombatingGerrymanderingRCV.PartyApprovalBallot](#)
- [GGRS26CombatingGerrymanderingRCV.paper_pav_thiele_weights](#)
- [GGRS26CombatingGerrymanderingRCV.paper_thiele_fixed_size_selected_committee](#)

Recorded source-to-declaration screening Verdict: matches

Reason: Specializes fixed-cardinality global Thiele maximization to harmonic PAV weights and the supplied tie rule, matching the source PAV winner definition.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GGRS26CombatingGerrymanderingRCV.paper_thiele_party_min_argmax

Paper-local declaration:

GGRS26CombatingGerrymanderingRCV.paper_thiele_party_min_argmax

Source locator: cited publication, lines 428-456

Verbatim paper-source connection

Calculating seat share given vote shares. Suppose – for a given district with N_k seats and V_k voters – we know the fraction y_p of voters that belong to each party $p \in \{R, D\}$ (vote share). How do we calculate the number of winners n_p from each party for a given social choice function?

Given our assumptions (including solid coalitions), it is straightforward to efficiently do so for Thiele rules, as candidates from a given party receive the same votes—and so we do not have to consider individual candidates, only the number of winners from each party directly. Then, we have that the party R seat share is:

$$n_R(y_R, \lambda) = \min \arg \max_X y_R \sum_{i=1}^N \lambda(i) + (1 - y_R) \sum_{i=1}^{N - n_R} \lambda(i) \quad (1)$$

The inner arg max comes directly from the rule definition – for a given n , suppose a fraction y_R approves n candidates. Then that will yield $y_R \sum_{i=1}^n \lambda(i)$ points, with the remaining fraction

$1 - y_R$ approving $N - n$ candidates and yielding an additional $(1 - y_R) \sum_{i=n+1}^N \lambda(i)$ points. The

outer min simply encodes tie-breaking in favor of party D.8 The party D seat share is $n_D(y_R, \lambda) \triangleq N - n_R(y_R, \lambda)$.

Lean-expanded paper semantic target

type:

GGRS26CombatingGerrymanderingRCV.PaperThieleWeights $\rightarrow \mathbb{N} \rightarrow \mathbb{R} \rightarrow \mathbb{N} \rightarrow \text{Prop}$

value:

```
fun (weights : GGRS26CombatingGerrymanderingRCV.PaperThieleWeights) (seatCount : ℕ) (partyShare : ℝ) (seats : ℕ) =>
  AppliedModelingLib.SocialChoice.Voting.IsMinArgmaxOn
    (GGRS26CombatingGerrymanderingRCV.paper_thiele_party_seat_score weights partyShare seats) seatCount seats
```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Voting.IsMinArgmaxOn](#)
- [GGRS26CombatingGerrymanderingRCV.PaperThieleWeights](#)
- [GGRS26CombatingGerrymanderingRCV.paper_thiele_party_seat_score](#)

Recorded source-to-declaration screening Verdict: matches

Reason: Uses the bounded global party-score maximum and chooses the smallest co-maximizer, exactly the source outer min arg max that favors D in party ties.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GGRS26CombatingGerrymanderingRCV.paper_thiele_party_republican_selection

Paper-local declaration:

GGRS26CombatingGerrymanderingRCV.paper_thiele_party_republican_selection

Source locator: cited publication, lines 428-456

Verbatim paper-source connection

Calculating seat share given vote shares. Suppose – for a given district with N_k seats and V_k voters – we know the fraction y_p of voters that belong to each party $p \in \{R, D\}$ (vote share). How do we calculate the number of winners n_p from each party for a given social choice function?

Given our assumptions (including solid coalitions), it is straightforward to efficiently do so for Thiele rules, as candidates from a given party receive the same votes—and so we do not have to consider individual candidates, only the number of winners from each party directly. Then, we have that the party R seat share is:

$$n_R(y_R, \lambda) = \min \arg \max_X y_R \sum_{i=1}^N \lambda(i) + (1 - y_R) \sum_{i=1}^{N - n_R} \lambda(i) \quad (1)$$

The inner arg max comes directly from the rule definition – for a given n , suppose a fraction y_R approves n candidates. Then that will yield $y_R \sum_{i=1}^n \lambda(i)$ points, with the remaining fraction

$1 - y_R$ approving $N - n$ candidates and yielding an additional $(1 - y_R) \sum_{i=n+1}^N \lambda(i)$ points. The

outer min simply encodes tie-breaking in favor of party D.8 The party D seat share is $n_D(y_R, \lambda) \triangleq N - n_R(y_R, \lambda)$.

Lean-expanded paper semantic target

```
type:
(weights : GGRS26CombatingGerrymanderingRCV.PaperThieleWeights) →
(parties : ℝ) →
(seats : ℕ) →
{ seatCount : ℕ //
  GGRS26CombatingGerrymanderingRCV.paper_thiele_party_min_argmax weights seatCount parties }

value:
fun (weights : GGRS26CombatingGerrymanderingRCV.PaperThieleWeights) (parties : ℝ) (seats : ℕ) =>
(Classical.choose
  (GGRS26CombatingGerrymanderingRCV.paper_thiele_party_republican_selection._proof_1 weights parties seats),
  GGRS26CombatingGerrymanderingRCV.paper_thiele_party_republican_selection._proof_2 weights parties seats)
```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Voting.IsMinArgmaxOn](#)
- [GGRS26CombatingGerrymanderingRCV.PaperThieleWeights](#)
- [GGRS26CombatingGerrymanderingRCV.paper_thiele_party_min_argmax](#)
- [GGRS26CombatingGerrymanderingRCV.paper_thiele_party_seat_score](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The selection subtype carries exactly the bounded maximum and smallest-tie conditions of the source n_R definition, without adding implementation semantics.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GGRS26CombatingGerrymanderingRCV.paper_thiele_party_republican_seat_count

Paper-local declaration:

GGRS26CombatingGerrymanderingRCV.paper_thiele_party_republican_seat_count

Source locator: cited publication, lines 428-456

Verbatim paper-source connection

Calculating seat share given vote shares. Suppose – for a given district with N_k seats and V_k voters – we know the fraction y_p of voters that belong to each party $p \in \{R, D\}$ (vote share). How do we calculate the number of winners n_p from each party for a given social choice function?

Given our assumptions (including solid coalitions), it is straightforward to efficiently do so for Thiele rules, as candidates from a given party receive the same votes—and so we do not have to consider individual candidates, only the number of winners from each party directly. Then, we have that the party R seat share is:

$$n_R(y_R, \lambda) = \min \arg \max_X \sum_{i=1}^N y_R \lambda(i) + (1 - y_R) \sum_{i=1}^{N - n} \lambda(i) \quad (1)$$

The inner arg max comes directly from the rule definition – for a given n , suppose a fraction y_R approves n candidates. Then that will yield $y_R \sum_{i=1}^n \lambda(i)$ points, with the remaining fraction

$1 - y_R$ approving $N - n$ candidates and yielding an additional $(1 - y_R) \sum_{i=1}^{N - n} \lambda(i)$ points. The

outer min simply encodes tie-breaking in favor of party D.8 The party D seat share is $n_D(y_R, \lambda) \triangleq N - n_R(y_R, \lambda)$.

Lean-expanded paper semantic target

type:
GGRS26CombatingGerrymanderingRCV.PaperThieleWeights $\rightarrow \mathbb{R} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$

value:
fun (weights : GGRS26CombatingGerrymanderingRCV.PaperThieleWeights) (partyShare : $\mathbb{R}$) (seats : $\mathbb{N}$) =>
↑(GGRS26CombatingGerrymanderingRCV.paper_thiele_party_republican_selection weights partyShare seats)

Direct retained dependencies

- [GGRS26CombatingGerrymanderingRCV.PaperThieleWeights](#)
- [GGRS26CombatingGerrymanderingRCV.paper_thiele_party_min_argmax](#)
- [GGRS26CombatingGerrymanderingRCV.paper_thiele_party_republican_selection](#)

Recorded source-to-declaration screening Verdict: matches

Reason: Projects the Republican component of the source least-maximizer selection while retaining the feasibility, maximality, and tie-breaking properties in the paired specification.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GGRS26CombatingGerrymanderingRCV.paper_pav_selected_seat_count

Paper-local declaration:

GGRS26CombatingGerrymanderingRCV.paper_pav_selected_seat_count

Source locator: cited publication, lines 428-456

Verbatim paper-source connection

Calculating seat share given vote shares. Suppose – for a given district with N_k seats and V_k voters – we know the fraction y_p of voters that belong to each party $p \in \{R, D\}$ (vote share). How do we calculate the number of winners n_p from each party for a given social choice function?

Given our assumptions (including solid coalitions), it is straightforward to efficiently do so for Thiele rules, as candidates from a given party receive the same votes—and so we do not have to consider individual candidates, only the number of winners from each party directly. Then, we have that the party R seat share is:

$$n_R(y_R, \lambda) = \min \arg \max_X \sum_{i=1}^N y_R \lambda(i) + (1 - y_R) \sum_{i=1}^{N - n} \lambda(i) \quad (1)$$

The inner arg max comes directly from the rule definition – for a given n , suppose a fraction y_R approves n candidates. Then that will yield $y_R \sum_{i=1}^n \lambda(i)$ points, with the remaining fraction

$1 - y_R$ approving $N - n$ candidates and yielding an additional $(1 - y_R) \sum_{i=n+1}^N \lambda(i)$ points. The

outer min simply encodes tie-breaking in favor of party D.8 The party D seat share is $n_D(y_R, \lambda) \triangleq N - n_R(y_R, \lambda)$.

Lean-expanded paper semantic target

type:
 $\mathbb{R} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$

value:
fun (partyShare : $\mathbb{R}$) (seats : $\mathbb{N}$) =>
GGRS26CombatingGerrymanderingRCV.paper_thiele_party_republican_seat_count
GGRS26CombatingGerrymanderingRCV.paper_pav_thiele_weights partyShare seats

Direct retained dependencies

- [GGRS26CombatingGerrymanderingRCV.paper_pav_thiele_weights](#)
- [GGRS26CombatingGerrymanderingRCV.paper_thiele_party_republican_seat_count](#)

Recorded source-to-declaration screening Verdict: matches

Reason: Projects the constructed PAV least-maximizing R-seat count, the source n_R selector used in Proposition 1.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GGRS26CombatingGerrymanderingRCV.paper_pav_selected_seat_count_spec

Paper-local declaration:

GGRS26CombatingGerrymanderingRCV.paper_pav_selected_seat_count_spec

Source locator: cited publication, lines 428-456

Verbatim paper-source connection

Calculating seat share given vote shares. Suppose – for a given district with N_k seats and V_k voters – we know the fraction y_p of voters that belong to each party $p \in \{R, D\}$ (vote share). How do we calculate the number of winners n_p from each party for a given social choice function?

Given our assumptions (including solid coalitions), it is straightforward to efficiently do so for Thiele rules, as candidates from a given party receive the same votes—and so we do not have to consider individual candidates, only the number of winners from each party directly. Then, we have that the party R seat share is:

$$n_R(y_R, \lambda) = \min \arg \max_X y_R \sum_{i=1}^N \lambda(i) + (1 - y_R) \sum_{i=1}^{N - n} \lambda(i) \quad (1)$$

The inner arg max comes directly from the rule definition – for a given n , suppose a fraction y_R approves n candidates. Then that will yield $y_R \sum_{i=1}^n \lambda(i)$ points, with the remaining fraction

$1 - y_R$ approving $N - n$ candidates and yielding an additional $(1 - y_R) \sum_{i=n+1}^N \lambda(i)$ points. The

outer min simply encodes tie-breaking in favor of party D.8 The party D seat share is $n_D(y_R, \lambda) \triangleq N - n_R(y_R, \lambda)$.

Lean-elaborated paper signature

$\forall$ (partyShare : $\mathbb{R}$) (seats : $\mathbb{N}$),
GGRS26CombatingGerrymanderingRCV.paper_pav_min_argmax
(GGRS26CombatingGerrymanderingRCV.paper_pav_selected_seat_count partyShare seats) partyShare seats

Direct retained dependencies

- [GGRS26CombatingGerrymanderingRCV.paper_pav_min_argmax](#)
- [GGRS26CombatingGerrymanderingRCV.paper_pav_selected_seat_count](#)

Recorded source-to-declaration screening Verdict: matches

Reason: States that the constructed PAV R-seat count satisfies the full bounded minimum-argmax selector relation from the source display.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GGRS26CombatingGerrymanderingRCV.paper_source_stv_execution_terminal

Paper-local declaration:

GGRS26CombatingGerrymanderingRCV.paper_source_stv_execution_terminal

Source locator: cited publication, lines 394-408

Verbatim paper-source connection

In Single Transferable Vote (STV), a type of ranked choice voting for multiple winners, each voter instead submits a ranking over candidates (here, we assume the rankings are complete, not partial). Consider a district with N_k seats and V_k voters; the set of winners is created iteratively, as follows. The number of first-place votes for each candidate is counted. Any candidate with a number of votes at least the “Droop” threshold $Q = \lfloor N/V_k \rfloor + 1$ is selected as a winner, and their surplus votes (number of votes minus Q) are transferred to each voter’s next preference. If no candidate has enough votes, then the candidate with the least number of votes (with tie-breaking) is eliminated, and their votes are transferred. The process continues until N_k candidates have been selected, or the number of remaining candidates equals the number of remaining seats. STV is commonly used in practice in multi-winner elections.

Details can differ in how such votes are transferred, but our results in Section 3 do not depend on the transfer rule; in our simulations in Section 4 and Appendix B, we use the “fractional” STV rule, in which each voter keeps a fractional number of votes equal to their share of the winning candidate’s surplus votes (we describe this formally in Section 4). For consistency, for all rules we assume that ties are broken in favor of candidates from party D, but randomly within each party.⁵

Lean-expanded paper semantic target

type:

```
{Voter : Type u_1} →  
{Candidate : Type u_2} → ℕ → GGRS26CombatingGerrymanderingRCV.PaperSourceSTVState Voter Candidate → Prop
```

value:

```
fun {Voter : Type u_1} {Candidate : Type u_2} (seats : ℕ)  
  (state : GGRS26CombatingGerrymanderingRCV.PaperSourceSTVState Voter Candidate) =>  
  state.elected.card = seats v state.active.card = seats - state.elected.card
```

Direct retained dependencies

- [GGRS26CombatingGerrymanderingRCV.PaperSourceSTVState](#)

Recorded source-to-declaration screening Verdict: matches

Reason: States precisely the source terminal cases: M elected candidates, or the still-active candidates fill the seats that remain.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GGRS26CombatingGerrymanderingRCV.paper_source_stv_tally

Paper-local declaration:

GGRS26CombatingGerrymanderingRCV.paper_source_stv_tally

Source locator: cited publication, lines 394-408

Verbatim paper-source connection

In Single Transferable Vote (STV), a type of ranked choice voting for multiple winners, each voter instead submits a ranking over candidates (here, we assume the rankings are complete, not partial). Consider a district with N_k seats and V_k voters; the set of winners is created iteratively, as follows. The number of first-place votes for each candidate is counted. Any candidate with a number of votes at least the “Droop” threshold $Q = \lfloor N_k k+1 \rfloor + 1$ is selected as a winner, and their surplus votes (number of votes minus Q) are transferred to each voter’s next preference. If no candidate has enough votes, then the candidate with the least number of votes (with tie-breaking) is eliminated, and their votes are transferred. The process continues until N_k candidates have been selected, or the number of remaining candidates equals the number of remaining seats. STV is commonly used in practice in multi-winner elections.

Details can differ in how such votes are transferred, but our results in Section 3 do not depend on the transfer rule; in our simulations in Section 4 and Appendix B, we use the “fractional” STV rule, in which each voter keeps a fractional number of votes equal to their share of the winning candidate’s surplus votes (we describe this formally in Section 4). For consistency, for all rules we assume that ties are broken in favor of candidates from party D, but randomly within each party.⁵

Lean-expanded paper semantic target

type:

```
{Voter : Type u_1} →  
{Candidate : Type u_2} →  
  Finset Voter →  
    (Voter → AppliedModelingLib.SocialChoice.Voting.Ballot Candidate) → Finset Candidate → (Voter → ℝ) → Candidate → ℝ
```

value:

```
fun {Voter : Type u_1} {Candidate : Type u_2} (allVoters : Finset Voter)  
  (ballots : Voter → AppliedModelingLib.SocialChoice.Voting.Ballot Candidate) (active : Finset Candidate)  
  (weight : Voter → ℝ) (candidate : Candidate) =>  
  Σ voter ∈ allVoters with  
    GGRS26CombatingGerrymanderingRCV.paper_first_active_candidate (ballots voter) active candidate, weight voter
```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Voting.Ballot](#)
- [GGRS26CombatingGerrymanderingRCV.paper_first_active_candidate](#)

Recorded source-to-declaration screening Verdict: matches

Reason: Sums the current weights of exactly those ballots whose first active listed candidate is the candidate being tallied, matching the source current tally.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GGRS26CombatingGerrymanderingRCV.paper_source_stv_d_favoring_transition

Paper-local declaration:

GGRS26CombatingGerrymanderingRCV.paper_source_stv_d_favoring_transition

Source locator: cited publication, lines 394-408

Verbatim paper-source connection

In Single Transferable Vote (STV), a type of ranked choice voting for multiple winners, each voter instead submits a ranking over candidates (here, we assume the rankings are complete, not partial). Consider a district with N_k seats and V_k voters; the set of winners is created iteratively, as follows. The number of first-place votes for each candidate is counted. Any candidate with a number of votes at least the “Droop” threshold $Q = \lfloor NV_k k+1 \rfloor + 1$ is selected as a winner, and their surplus votes (number of votes minus Q) are transferred to each voter’s next preference. If no candidate has enough votes, then the candidate with the least number of votes (with tie-breaking) is eliminated, and their votes are transferred. The process continues until N_k candidates have been selected, or the number of remaining candidates equals the number of remaining seats. STV is commonly used in practice in multi-winner elections.

Details can differ in how such votes are transferred, but our results in Section 3 do not depend on the transfer rule; in our simulations in Section 4 and Appendix B, we use the “fractional” STV rule, in which each voter keeps a fractional number of votes equal to their share of the winning candidate’s surplus votes (we describe this formally in Section 4). For consistency, for all rules we assume that ties are broken in favor of candidates from party D, but randomly within each party.⁵

Lean-expanded paper semantic target

```
type:
{Voter : Type u_1} →
{Candidate : Type u_2} →
  Finset Voter →
    (Voter → AppliedModelingLib.SocialChoice.Voting.Ballot Candidate) →
      Finset Candidate →
        ℝ →
          ℕ →
            (Finset Candidate → Candidate → (Voter → ℝ) → (Voter → ℝ) → Prop) →
              (Finset Candidate → Candidate → (Voter → ℝ) → (Voter → ℝ) → Prop) →
                GGRS26CombatingGerrymanderingRCV.PaperSourceSTVState Voter Candidate →
                  GGRS26CombatingGerrymanderingRCV.PaperSourceSTVState Voter Candidate → Prop

value:
fun {Voter : Type u_1} {Candidate : Type u_2} (allVoters : Finset Voter)
  (ballots : Voter → AppliedModelingLib.SocialChoice.Voting.Ballot Candidate) (favoredParty : Finset Candidate)
  (quota : ℝ) (seats : ℕ)
  (electUpdate eliminateUpdate : Finset Candidate → Candidate → (Voter → ℝ) → (Voter → ℝ) → Prop)
  (before after : GGRS26CombatingGerrymanderingRCV.PaperSourceSTVState Voter Candidate) =>
  (∃ (winner : Candidate),
    →GGRS26CombatingGerrymanderingRCV.paper_source_stv_execution_terminal seats before ∧
    winner ∈ before.active ∧
    before.elected.card < seats ∧
    quota ≤
      GGRS26CombatingGerrymanderingRCV.paper_source_stv_tally allVoters ballots before.active before.weight
    winner ∧
    (winner ∉ favoredParty →
      ∀ candidate ∈ before.active,
        GGRS26CombatingGerrymanderingRCV.paper_source_stv_tally allVoters ballots before.active
        before.weight candidate =
          GGRS26CombatingGerrymanderingRCV.paper_source_stv_tally allVoters ballots before.active
        before.weight winner →
        candidate ∉ favoredParty) ∧
    after.active = before.active.erase winner ∧
    after.elected = insert winner before.elected ∧
    electUpdate before.active winner before.weight after.weight) ∧
  (∃ (loser : Candidate),
    →GGRS26CombatingGerrymanderingRCV.paper_source_stv_execution_terminal seats before ∧
    loser ∈ before.active ∧
    (∀ candidate ∈ before.active,
      GGRS26CombatingGerrymanderingRCV.paper_source_stv_tally allVoters ballots before.active before.weight
      candidate <
        quota) ∧
    (∀ candidate ∈ before.active,
      GGRS26CombatingGerrymanderingRCV.paper_source_stv_tally allVoters ballots before.active before.weight
      loser ≤
        GGRS26CombatingGerrymanderingRCV.paper_source_stv_tally allVoters ballots before.active before.weight
      candidate) ∧
    (loser ∈ favoredParty →
      ∀ candidate ∈ before.active,
        GGRS26CombatingGerrymanderingRCV.paper_source_stv_tally allVoters ballots before.active
        before.weight candidate =
          GGRS26CombatingGerrymanderingRCV.paper_source_stv_tally allVoters ballots before.active
        before.weight loser →
        candidate ∈ favoredParty) ∧
    after.active = before.active.erase loser ∧
    after.elected = before.elected ∧ eliminateUpdate before.active loser before.weight after.weight)
```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Voting.Ballot](#)
- [GGRS26CombatingGerrymanderingRCV.PaperSourceSTVState](#)

- [GGRS26CombattingGerrymanderingRCV.paper_source_stv_execution_terminal](#)
- [GGRS26CombattingGerrymanderingRCV.paper_source_stv_tally](#)

Recorded source-to-declaration screening Verdict: matches

Reason: Gives the source-shaped quota-election-before-elimination and minimum-tally elimination cases, favors D in cross-party ties, and represents all within-party random outcomes as pathwise support only.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GGRS26CombatingGerrymanderingRCV.paper_source_surplus_transfer_policy

Paper-local declaration:

GGRS26CombatingGerrymanderingRCV.paper_source_surplus_transfer_policy

Source locator: cited publication, lines 394-408

Verbatim paper-source connection

In Single Transferable Vote (STV), a type of ranked choice voting for multiple winners, each voter instead submits a ranking over candidates (here, we assume the rankings are complete, not partial). Consider a district with N_k seats and V_k voters; the set of winners is created iteratively, as follows. The number of first-place votes for each candidate is counted. Any candidate with a number of votes at least the “Droop” threshold $Q = \lfloor NV_k / k + 1 \rfloor + 1$ is selected as a winner, and their surplus votes (number of votes minus Q) are transferred to each voter’s next preference. If no candidate has enough votes, then the candidate with the least number of votes (with tie-breaking) is eliminated, and their votes are transferred. The process continues until N_k candidates have been selected, or the number of remaining candidates equals the number of remaining seats. STV is commonly used in practice in multi-winner elections.

Details can differ in how such votes are transferred, but our results in Section 3 do not depend on the transfer rule; in our simulations in Section 4 and Appendix B, we use the “fractional” STV rule, in which each voter keeps a fractional number of votes equal to their share of the winning candidate’s surplus votes (we describe this formally in Section 4). For consistency, for all rules we assume that ties are broken in favor of candidates from party D, but randomly within each party.⁵

Lean-expanded paper semantic target

type:

```
{Voter : Type u_1} →  
{Candidate : Type u_2} →  
  Finset Voter →  
    (Voter → AppliedModelingLib.SocialChoice.Voting.Ballot Candidate) →  
      ℝ →  
        (Finset Candidate → Candidate → (Voter → ℝ) → (Voter → ℝ) → Prop) →  
          (Finset Candidate → Candidate → (Voter → ℝ) → (Voter → ℝ) → Prop) → Prop
```

value:

```
fun {Voter : Type u_1} {Candidate : Type u_2} (allVoters : Finset Voter)  
  (ballots : Voter → AppliedModelingLib.SocialChoice.Voting.Ballot Candidate) (quota : ℝ)  
  (electUpdate eliminateUpdate : Finset Candidate → Candidate → (Voter → ℝ) → (Voter → ℝ) → Prop) =>  
  (∀ (active : Finset Candidate) (winner : Candidate) (beforeWeight : Voter → ℝ),  
    (∀ voter ∈ allVoters, 0 ≤ beforeWeight voter) →  
      winner ∈ active →  
        quota ≤  
          GGRS26CombatingGerrymanderingRCV.paper_source_stv_tally allVoters ballots active beforeWeight winner →  
            ∃ (afterWeight : Voter → ℝ), electUpdate active winner beforeWeight afterWeight) ∧  
    (∀ (active : Finset Candidate) (loser : Candidate) (beforeWeight : Voter → ℝ),  
      (∀ voter ∈ allVoters, 0 ≤ beforeWeight voter) →  
        loser ∈ active → ∃ (afterWeight : Voter → ℝ), eliminateUpdate active loser beforeWeight afterWeight) ∧  
    (∀ (active : Finset Candidate) (winner : Candidate) (beforeWeight afterWeight : Voter → ℝ),  
      (∀ voter ∈ allVoters, 0 ≤ beforeWeight voter) →  
        electUpdate active winner beforeWeight afterWeight → ∀ voter ∈ allVoters, 0 ≤ afterWeight voter) ∧  
    (∀ (active : Finset Candidate) (winner : Candidate) (beforeWeight afterWeight : Voter → ℝ),  
      (∀ voter ∈ allVoters, 0 ≤ beforeWeight voter) →  
        electUpdate active winner beforeWeight afterWeight →  
          ∀ voter ∈ allVoters,  
            ¬GGRS26CombatingGerrymanderingRCV.paper_first_active_candidate (ballots voter) active winner →  
              afterWeight voter = beforeWeight voter) ∧  
    (∀ (active : Finset Candidate) (winner : Candidate) (beforeWeight afterWeight : Voter → ℝ),  
      (∀ voter ∈ allVoters, 0 ≤ beforeWeight voter) →  
        electUpdate active winner beforeWeight afterWeight →  
          ∑ voter ∈ allVoters with  
            GGRS26CombatingGerrymanderingRCV.paper_first_active_candidate (ballots voter) active winner,  
            afterWeight voter =  
              ∑ voter ∈ allVoters with  
                GGRS26CombatingGerrymanderingRCV.paper_first_active_candidate (ballots voter) active winner,  
                beforeWeight voter -  
                  quota) ∧  
    ∀ (active : Finset Candidate) (loser : Candidate) (beforeWeight afterWeight : Voter → ℝ),  
      (∀ voter ∈ allVoters, 0 ≤ beforeWeight voter) →  
        eliminateUpdate active loser beforeWeight afterWeight →  
          ∀ voter ∈ allVoters, afterWeight voter = beforeWeight voter
```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Voting.Ballot](#)
- [GGRS26CombatingGerrymanderingRCV.paper_first_active_candidate](#)
- [GGRS26CombatingGerrymanderingRCV.paper_source_stv_tally](#)

Recorded source-to-declaration screening Verdict: matches

Reason: Requires a quota winner’s supporter mass to fall by exactly the quota while off-support mass is fixed and leaves eliminations unchanged, matching the source’s stated surplus-preserving transfer-policy scope.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

--

GGRS26CombatingGerrymanderingRCV.paper_source_stv_rule_execution

Paper-local declaration:

GGRS26CombatingGerrymanderingRCV.paper_source_stv_rule_execution

Source locator: cited publication, lines 394-408

Verbatim paper-source connection

In Single Transferable Vote (STV), a type of ranked choice voting for multiple winners, each voter instead submits a ranking over candidates (here, we assume the rankings are complete, not partial). Consider a district with N_k seats and V_k voters; the set of winners is created iteratively, as follows. The number of first-place votes for each candidate is counted. Any candidate with a number of votes at least the “Droop” threshold $Q = \lfloor NV_k \frac{k+1}{k+1} \rfloor + 1$ is selected as a winner, and their surplus votes (number of votes minus Q) are transferred to each voter’s next preference. If no candidate has enough votes, then the candidate with the least number of votes (with tie-breaking) is eliminated, and their votes are transferred. The process continues until N_k candidates have been selected, or the number of remaining candidates equals the number of remaining seats. STV is commonly used in practice in multi-winner elections.

Details can differ in how such votes are transferred, but our results in Section 3 do not depend on the transfer rule; in our simulations in Section 4 and Appendix B, we use the “fractional” STV rule, in which each voter keeps a fractional number of votes equal to their share of the winning candidate’s surplus votes (we describe this formally in Section 4). For consistency, for all rules we assume that ties are broken in favor of candidates from party D, but randomly within each party.⁵

Lean-expanded paper semantic target

```
type:
{Voter : Type u_1} →
{Candidate : Type u_2} →
  Finset Voter →
    (Voter → AppliedModelingLib.SocialChoice.Voting.Ballot Candidate) →
      Finset Candidate →
        Finset Candidate →
          ℕ →
            (Finset Candidate → Candidate → (Voter → ℝ) → (Voter → ℝ) → Prop) →
              (Finset Candidate → Candidate → (Voter → ℝ) → (Voter → ℝ) → Prop) →
                GGRS26CombatingGerrymanderingRCV.PaperSourceSTVState Voter Candidate → Prop

value:
fun {Voter : Type u_1} {Candidate : Type u_2} (allVoters : Finset Voter)
  (ballots : Voter → AppliedModelingLib.SocialChoice.Voting.Ballot Candidate)
  (initialCandidates favoredParty : Finset Candidate) (seats : ℕ)
  (electUpdate eliminateUpdate : Finset Candidate → Candidate → (Voter → ℝ) → (Voter → ℝ) → Prop)
  (terminal : GGRS26CombatingGerrymanderingRCV.PaperSourceSTVState Voter Candidate) =>
have initial : GGRS26CombatingGerrymanderingRCV.PaperSourceSTVState Voter Candidate :=
{ active := initialCandidates, elected := ∅, weight := fun (x : Voter) => 1 };
(∀ voter ∈ allVoters,
  (ballots voter).Valid ∧ ∀ (candidate : Candidate), candidate ∈ ballots voter ↔ candidate ∈ initialCandidates) ∧
favoredParty ⊆ initialCandidates ∧
GGRS26CombatingGerrymanderingRCV.paper_source_surplus_transfer_policy allVoters ballots
(↑ (AppliedModelingLib.SocialChoice.Voting.STVQuota seats allVoters.card)) electUpdate eliminateUpdate ∧
Relation.ReflTransGen
  (GGRS26CombatingGerrymanderingRCV.paper_source_stv_d_favoring_transition allVoters ballots favoredParty
    (↑ (AppliedModelingLib.SocialChoice.Voting.STVQuota seats allVoters.card)) seats electUpdate
    eliminateUpdate)
initial terminal ∧
GGRS26CombatingGerrymanderingRCV.paper_source_stv_execution_terminal seats terminal
```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Voting.Ballot](#)
- [AppliedModelingLib.SocialChoice.Voting.Ballot.Valid](#)
- [AppliedModelingLib.SocialChoice.Voting.STVQuota](#)
- [GGRS26CombatingGerrymanderingRCV.PaperSourceSTVState](#)
- [GGRS26CombatingGerrymanderingRCV.paper_source_stv_d_favoring_transition](#)
- [GGRS26CombatingGerrymanderingRCV.paper_source_stv_execution_terminal](#)
- [GGRS26CombatingGerrymanderingRCV.paper_source_surplus_transfer_policy](#)

Recorded source-to-declaration screening Verdict: matches

Reason: Combines complete valid rankings, exact Droop quota, initial unit weights, a surplus-transfer policy, D-favoring transitions, and a terminal state; this is the selected source execution route and asserts no probability law.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GGRS26CombatingGerrymanderingRCV.paper_thiele_party_democratic_seat_count

Paper-local declaration:

GGRS26CombatingGerrymanderingRCV.paper_thiele_party_democratic_seat_count

Source locator: cited publication, lines 428-456

Verbatim paper-source connection

Calculating seat share given vote shares. Suppose – for a given district with N_k seats and V_k voters – we know the fraction y_p of voters that belong to each party $p \in \{R, D\}$ (vote share). How do we calculate the number of winners n_p from each party for a given social choice function?

Given our assumptions (including solid coalitions), it is straightforward to efficiently do so for Thiele rules, as candidates from a given party receive the same votes—and so we do not have to consider individual candidates, only the number of winners from each party directly. Then, we have that the party R seat share is:

$$n_R(y_R, \lambda) = \min \arg \max_X \sum_{i=1}^N y_R \lambda(i) + (1 - y_R) \sum_{i=1}^{N - n_R} \lambda(i) \quad (1)$$

The inner $\arg \max$ comes directly from the rule definition – for a given n , suppose a fraction y_R approves n candidates. Then that will yield $y_R \sum_{i=1}^n \lambda(i)$ points, with the remaining fraction

$1 - y_R$ approving $N - n$ candidates and yielding an additional $(1 - y_R) \sum_{i=n+1}^N \lambda(i)$ points. The

outer $\min$ simply encodes tie-breaking in favor of party D . The party D seat share is $n_D(y_R, \lambda) \triangleq N - n_R(y_R, \lambda)$.

Lean-expanded paper semantic target

type:
GGRS26CombatingGerrymanderingRCV.PaperThieleWeights $\rightarrow \mathbb{R} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$

value:
fun (weights : GGRS26CombatingGerrymanderingRCV.PaperThieleWeights) (partyShare : $\mathbb{R}$) (seats : $\mathbb{N}$) =>
seats - GGRS26CombatingGerrymanderingRCV.paper_thiele_party_republican_seat_count weights partyShare seats

Direct retained dependencies

- [GGRS26CombatingGerrymanderingRCV.PaperThieleWeights](#)
- [GGRS26CombatingGerrymanderingRCV.paper_thiele_party_republican_seat_count](#)

Recorded source-to-declaration screening Verdict: matches

Reason: Defines the Democratic count as M minus the selected Republican count, exactly the source relation $n_D = M - n_R$.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GGRS26CombatingGerrymanderingRCV.paper_thiele_party_republican_seat_count_spec

Paper-local declaration:

GGRS26CombatingGerrymanderingRCV.paper_thiele_party_republican_seat_count_spec

Source locator: cited publication, lines 428-456

Verbatim paper-source connection

Calculating seat share given vote shares. Suppose – for a given district with N_k seats and V_k voters – we know the fraction y_p of voters that belong to each party $p \in \{R, D\}$ (vote share). How do we calculate the number of winners n_p from each party for a given social choice function?

Given our assumptions (including solid coalitions), it is straightforward to efficiently do so for Thiele rules, as candidates from a given party receive the same votes—and so we do not have to consider individual candidates, only the number of winners from each party directly. Then, we have that the party R seat share is:

$$n_R(y_R, \lambda) = \min_p \arg \max_X \sum_{i=1}^N y_R \lambda(i) + (1 - y_R) \sum_{i=1}^{N - n} \lambda(i) \quad (1)$$

The inner arg max comes directly from the rule definition – for a given n , suppose a fraction y_R approves n candidates. Then that will yield $\sum_{i=1}^n y_R \lambda(i)$ points, with the remaining fraction

$1 - y_R$ approving $N - n$ candidates and yielding an additional $\sum_{i=1}^{N - n} (1 - y_R) \lambda(i)$ points. The

outer min simply encodes tie-breaking in favor of party D.8 The party D seat share is $n_D(y_R, \lambda) \triangleq N - n_R(y_R, \lambda)$.

Lean-elaborated paper signature

$\forall$ (weights : GGRS26CombatingGerrymanderingRCV.PaperThieleWeights) (partyShare : $\mathbb{R}$) (seats : $\mathbb{N}$),
GGRS26CombatingGerrymanderingRCV.paper_thiele_party_min_argmax weights
(GGRS26CombatingGerrymanderingRCV.paper_thiele_party_republican_seat_count weights partyShare seats) partyShare
seats

Direct retained dependencies

- [GGRS26CombatingGerrymanderingRCV.PaperThieleWeights](#)
- [GGRS26CombatingGerrymanderingRCV.paper_thiele_party_min_argmax](#)
- [GGRS26CombatingGerrymanderingRCV.paper_thiele_party_republican_seat_count](#)

Recorded source-to-declaration screening Verdict: matches

Reason: States that the selected Republican count obeys the full source bounded least-maximizer relation for the two-party Thiele score.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GGRS26CombatingGerrymanderingRCV.paper_thiele_party_selected_seat_counts

Paper-local declaration:

GGRS26CombatingGerrymanderingRCV.paper_thiele_party_selected_seat_counts

Source locator: cited publication, lines 428-456

Verbatim paper-source connection

Calculating seat share given vote shares. Suppose – for a given district with N_k seats and V_k voters – we know the fraction y_p of voters that belong to each party $p \in \{R, D\}$ (vote share). How do we calculate the number of winners n_p from each party for a given social choice function?

Given our assumptions (including solid coalitions), it is straightforward to efficiently do so for Thiele rules, as candidates from a given party receive the same votes—and so we do not have to consider individual candidates, only the number of winners from each party directly. Then, we have that the party R seat share is:

$$n_R(y_R, \lambda) = \min \arg \max_X \sum_{i=1}^N y_R \lambda(i) + (1 - y_R) \sum_{i=1}^N \lambda(i) \quad (1)$$

The inner $\arg \max$ comes directly from the rule definition – for a given n , suppose a fraction y_R approves n candidates. Then that will yield $y_R \sum_{i=1}^N \lambda(i)$ points, with the remaining fraction

$1 - y_R$ approving $N - n$ candidates and yielding an additional $(1 - y_R) \sum_{i=1}^N \lambda(i)$ points. The

outer $\min$ simply encodes tie-breaking in favor of party D .⁸ The party D seat share is $n_D(y_R, \lambda) \triangleq N - n_R(y_R, \lambda)$.

Lean-expanded paper semantic target

type:
GGRS26CombatingGerrymanderingRCV.PaperThieleWeights $\rightarrow \mathbb{R} \rightarrow \mathbb{N} \rightarrow \mathbb{N} \rightarrow \text{Prop}$

value:
fun (weights : GGRS26CombatingGerrymanderingRCV.PaperThieleWeights) (partyShare : $\mathbb{R}$) (seats nR nD : $\mathbb{N}$) =>
GGRS26CombatingGerrymanderingRCV.paper_thiele_party_min_argmax weights nR partyShare seats $\wedge$ nD = seats - nR

Direct retained dependencies

- [GGRS26CombatingGerrymanderingRCV.PaperThieleWeights](#)
- [GGRS26CombatingGerrymanderingRCV.paper_thiele_party_min_argmax](#)

Recorded source-to-declaration screening Verdict: matches

Reason: Pairs the source least-maximizing Republican count with the complementary Democratic count, matching the two reported party seat shares.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GGRS26CombatingGerrymanderingRCV.paper_thiele_party_selected_seat_counts_add

Paper-local declaration:

GGRS26CombatingGerrymanderingRCV.paper_thiele_party_selected_seat_counts_add

Source locator: cited publication, lines 428-456

Verbatim paper-source connection

Calculating seat share given vote shares. Suppose – for a given district with N_k seats and V_k voters – we know the fraction y_p of voters that belong to each party $p \in \{R, D\}$ (vote share). How do we calculate the number of winners n_p from each party for a given social choice function?

Given our assumptions (including solid coalitions), it is straightforward to efficiently do so for Thiele rules, as candidates from a given party receive the same votes—and so we do not have to consider individual candidates, only the number of winners from each party directly. Then, we have that the party R seat share is:

$$n_R(y_R, \lambda) = \min_p \arg \max_X y_R \sum_{i=1}^N \lambda(i) + (1 - y_R) \sum_{i=1}^{N - n} \lambda(i) \quad (1)$$

The inner arg max comes directly from the rule definition – for a given n , suppose a fraction y_R approves n candidates. Then that will yield $y_R \sum_{i=1}^n \lambda(i)$ points, with the remaining fraction

$1 - y_R$ approving $N - n$ candidates and yielding an additional $(1 - y_R) \sum_{i=1}^{N - n} \lambda(i)$ points. The

outer min simply encodes tie-breaking in favor of party D.8 The party D seat share is $n_D(y_R, \lambda) \triangleq N - n_R(y_R, \lambda)$.

Lean-elaborated paper signature

$\forall$ (weights : GGRS26CombatingGerrymanderingRCV.PaperThieleWeights) (partyShare : $\mathbb{R}$) (seats : $\mathbb{N}$),
GGRS26CombatingGerrymanderingRCV.paper_thiele_party_republican_seat_count weights partyShare seats +
GGRS26CombatingGerrymanderingRCV.paper_thiele_party_democratic_seat_count weights partyShare seats =
seats

Direct retained dependencies

- [GGRS26CombatingGerrymanderingRCV.PaperThieleWeights](#)
- [GGRS26CombatingGerrymanderingRCV.paper_thiele_party_democratic_seat_count](#)
- [GGRS26CombatingGerrymanderingRCV.paper_thiele_party_republican_seat_count](#)

Recorded source-to-declaration screening Verdict: matches

Reason: Proves the paired counts sum to the district magnitude through the selected count's bound, expressing the source complement relation without natural-number truncation.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GGRS26CombatingGerrymanderingRCV.paper_thiele_party_selected_seat_counts_spec

Paper-local declaration:

GGRS26CombatingGerrymanderingRCV.paper_thiele_party_selected_seat_counts_spec

Source locator: cited publication, lines 428-456

Verbatim paper-source connection

Calculating seat share given vote shares. Suppose – for a given district with N_k seats and V_k voters – we know the fraction y_p of voters that belong to each party $p \in \{R, D\}$ (vote share). How do we calculate the number of winners n_p from each party for a given social choice function?

Given our assumptions (including solid coalitions), it is straightforward to efficiently do so for Thiele rules, as candidates from a given party receive the same votes—and so we do not have to consider individual candidates, only the number of winners from each party directly. Then, we have that the party R seat share is:

$$n_R(y_R, \lambda) = \min \arg \max_X y_R \sum_{i=1}^N \lambda(i) + (1 - y_R) \sum_{i=1}^N \lambda(i) \quad (1)$$

The inner arg max comes directly from the rule definition – for a given n , suppose a fraction y_R approves n candidates. Then that will yield $y_R \sum_{i=1}^N \lambda(i)$ points, with the remaining fraction

$1 - y_R$ approving $N - n$ candidates and yielding an additional $(1 - y_R) \sum_{i=1}^N \lambda(i)$ points. The

outer min simply encodes tie-breaking in favor of party D.8 The party D seat share is $n_D(y_R, \lambda) \triangleq N - n_R(y_R, \lambda)$.

Lean-elaborated paper signature

$\forall$ (weights : GGRS26CombatingGerrymanderingRCV.PaperThieleWeights) (partyShare : $\mathbb{R}$) (seats : $\mathbb{N}$),
GGRS26CombatingGerrymanderingRCV.paper_thiele_party_selected_seat_counts weights partyShare seats
(GGRS26CombatingGerrymanderingRCV.paper_thiele_party_republican_seat_count weights partyShare seats)
(GGRS26CombatingGerrymanderingRCV.paper_thiele_party_democratic_seat_count weights partyShare seats)

Direct retained dependencies

- [GGRS26CombatingGerrymanderingRCV.PaperThieleWeights](#)
- [GGRS26CombatingGerrymanderingRCV.paper_thiele_party_democratic_seat_count](#)
- [GGRS26CombatingGerrymanderingRCV.paper_thiele_party_republican_seat_count](#)
- [GGRS26CombatingGerrymanderingRCV.paper_thiele_party_selected_seat_counts](#)

Recorded source-to-declaration screening Verdict: matches

Reason: States the exact source selector relation together with the Democratic complement for the constructed pair of party seat counts.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GGRS26CombatingGerrymanderingRCV.paper_two_party_voter_model

Paper-local declaration:

GGRS26CombatingGerrymanderingRCV.paper_two_party_voter_model

Source locator: cited publication, lines 415-426

Verbatim paper-source connection

Voter assumptions. Next, we require assumptions on how voters rank or approve candidates. For each Thiele rule, we assume that in each district k there are exactly N_k candidates from each party, and each voter simply approves all the candidates in their party.⁶ For STV, we assume that in each district k there are at least N_k candidates from each party.⁷ For our primarily empirical analysis, we further assume that parties are solid coalitions (Dummett 1984): that each voter ranks all candidates of their party over each candidate of the other party, though intra-party rankings may vary. These assumptions reflect that party membership is a strong predictor of the party of the candidate for which a voter votes. Such an assumption, using historical party vote shares to predict vote shares in future elections, is used throughout the redistricting literature for SMDs; our additional assumption is that this behavior extends to partisan voting in multi-member elections. We relax this assumption in Section 4 and study intra-party effects, where voters may disagree on within-party rankings, in Appendix B.

Lean-expanded paper semantic target

```
type:
{Voter : Type u_1} →
{Candidate : Type u_2} →
  Finset Voter →
    Finset Voter →
      Finset Voter →
        Finset Candidate →
          Finset Candidate →
            (Voter → GGRS26CombatingGerrymanderingRCV.PartyApprovalBallot Candidate) →
              Finset Candidate →
                Finset Candidate →
                  Finset Candidate → (Voter → AppliedModelingLib.SocialChoice.Voting.Ballot Candidate) → ℕ → Prop
```

```
value:
fun {Voter : Type u_1} {Candidate : Type u_2} (allVoters partyVoters otherPartyVoters : Finset Voter)
  (thielePartyCandidates thieleOtherPartyCandidates : Finset Candidate)
  (approvalBallots : Voter → GGRS26CombatingGerrymanderingRCV.PartyApprovalBallot Candidate)
  (stvPartyCandidates stvOtherPartyCandidates allSTVCandidates : Finset Candidate)
  (rankingBallots : Voter → AppliedModelingLib.SocialChoice.Voting.Ballot Candidate) (seats : ℕ) =>
allVoters = partyVoters ∪ otherPartyVoters ∧
Disjoint partyVoters otherPartyVoters ∧
Disjoint thielePartyCandidates thieleOtherPartyCandidates ∧
thielePartyCandidates.card = seats ∧
thieleOtherPartyCandidates.card = seats ∧
(∀ voter ∈ partyVoters, approvalBallots voter = thielePartyCandidates) ∧
(∀ voter ∈ otherPartyVoters, approvalBallots voter = thieleOtherPartyCandidates) ∧
seats ≤ stvPartyCandidates.card ∧
seats ≤ stvOtherPartyCandidates.card ∧
GGRS26CombatingGerrymanderingRCV.paper_complete_solid_coalition_rankings partyVoters rankingBallots
stvPartyCandidates stvOtherPartyCandidates allSTVCandidates ∧
GGRS26CombatingGerrymanderingRCV.paper_complete_solid_coalition_rankings otherPartyVoters
rankingBallots stvOtherPartyCandidates stvPartyCandidates allSTVCandidates
```

Direct retained dependencies

- [AppliedModelingLib.SocialChoice.Voting.Ballot](#)
- [GGRS26CombatingGerrymanderingRCV.PartyApprovalBallot](#)
- [GGRS26CombatingGerrymanderingRCV.paper_complete_solid_coalition_rankings](#)

Recorded source-to-declaration screening Verdict: matches

Reason: Uses disjoint R/D voter blocks and candidate rosters, party approval sets, enough candidates for each rule, and complete solid-coalition rankings, matching the source two-party electorate and ballot model.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Source claims

Review row 1

Source locator: cited publication, lines 2414-2433

Verbatim source input

Lemma C.1. Let $yR \in (0, 1]$. Consider

$$nR(yR, \lambda PAV) = \min \arg \max_X \sum_{i=1}^n \lambda(i) + (1 - yR) \sum_{i=1}^n \lambda(i).$$

and let ell be the unique integer such that

$$yR(M + 1) - 1 \leq ell < yR(M + 1)$$

Then, $nR(yR, \lambda PAV) = ell$, for all yR, M .

[Next verbatim source excerpt]

A Thiele rule is characterized by a function λ . Each voter v in district k provides a set S_v of the N_k candidates they like most. The set of winners is determined as follows. Consider a potential set of winners C . The amount of points that voter v contributes to C is $\sum_{i=1}^{|S_v \cap C|} \lambda(i)$, and

the set C with the most points across voters (after tie-breaking) is selected. The (non-increasing) function λ establishes that votes have diminishing returns; the i th approved candidate in the set is worth $\lambda(i)$. For example, winner-take-all approval voting is a Thiele rule with $\lambda(i) = 1$: for each voter, the set C receives a number of points equal to the number of candidates in the set that the voter likes, and so the N_k candidates with the most votes win. We further consider Proportional Approval Voting (PAV), a Thiele rule with $\lambda PAV(i) = 1/i$; and Thiele Squared, with $\lambda TS = i/2$. PAV is well-studied in the theoretical social choice literature and comes with optimality guarantees (in terms of proportionality) (Skowron 2021). Thiele Squared induces even sharper diminishing returns than PAV in a voter having more of their preferred candidates winning, and thus prioritizes more voters having at least one approved candidate (and 50/50 outcomes between two parties, regardless of underlying vote shares). More generally, Thiele rules both contain common voting rules and parameterize a rule's tendency to favor proportional or balanced outcomes.

Expanded PaperInterface specification

```
∀ {seats : ℕ} {partyShare : ℝ},
  1 ≤ seats →
  0 < partyShare →
  partyShare ≤ 1 →
  ∃ (seatCount : ℕ),
    GGRS26CombattingGerrymanderingRCV.paper_pav_min_argmax seatCount partyShare seats ∧
    ∃ (ell : ℤ),
      ↑seatCount = ell ∧
      GGRS26CombattingGerrymanderingRCV.paper_pav_integer_interval ell partyShare seats ∧
      (∀ (other : ℤ),
        GGRS26CombattingGerrymanderingRCV.paper_pav_integer_interval other partyShare seats →
        other = ell) ∧
      ∀ (otherSeatCount : ℕ),
        GGRS26CombattingGerrymanderingRCV.paper_pav_min_argmax otherSeatCount partyShare seats →
        ↑otherSeatCount = ell
```

Retained governing declarations

- [GGRS26CombattingGerrymanderingRCV.paper_pav_integer_interval](#)
- [GGRS26CombattingGerrymanderingRCV.paper_pav_min_argmax](#)

Lean-elaborated claim roles

```
1. parameter: ℕ
2. parameter: ℝ
3. assumption: 1 ≤ seats
4. assumption: 0 < partyShare
5. assumption: partyShare ≤ 1
6. conclusion: ∃ (seatCount : ℕ),
  GGRS26CombattingGerrymanderingRCV.paper_pav_min_argmax seatCount partyShare seats ∧
  ∃ (ell : ℤ),
    ↑seatCount = ell ∧
    GGRS26CombattingGerrymanderingRCV.paper_pav_integer_interval ell partyShare seats ∧
    (∀ (other : ℤ),
      GGRS26CombattingGerrymanderingRCV.paper_pav_integer_interval other partyShare seats → other = ell) ∧
    ∀ (otherSeatCount : ℕ),
      GGRS26CombattingGerrymanderingRCV.paper_pav_min_argmax otherSeatCount partyShare seats →
      ↑otherSeatCount = ell
```

Lean proof endpoint (built separately)

GGRS26CombattingGerrymanderingRCV.paper_lemma_c1_pav_selector_eq_unique_integer_interval

Recorded source-to-Spec screening Verdict: matches

Reason: The frozen target has the source's positive party-share and positive district-magnitude domain, PAV harmonic leftmost maximizer on 0 through M, the displayed half-open integer interval, and equality of every such selector to its unique ell.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 2

Source locator: cited publication, lines 485-490

Verbatim source input

Proposition 1 (Seat shares under STV). Suppose – for a given district with M seats and $V \geq M(M + 1)$ voters – that a fraction y_p of voters belong to each party $p \in \{R, D\}$, and that there are at least M candidates per party. Assume that each party’s voters rank all same-party candidates above all other-party candidates, and ties are broken in party D’s favor.

Let $n_R(y_R, STV)$ be the number of winners belonging to party R using STV. Then, both $n_R(y_R, STV)$ and $n_R(y_R, \lambda PAV)$ are in $\{y_R M\}$.

[Next verbatim source excerpt]

In Single Transferable Vote (STV), a type of ranked choice voting for multiple winners, each voter instead submits a ranking over candidates (here, we assume the rankings are complete, not partial). Consider a district with N_k seats and V_k voters; the set of winners is created iteratively, as follows. The number of first-place votes for each candidate is counted. Any candidate with a number of votes at least the “Droop” threshold $Q = \lfloor \frac{V_k}{N_k + 1} \rfloor + 1$ is selected as a winner, and their surplus votes (number of votes minus Q) are transferred to each voter’s next preference. If no candidate has enough votes, then the candidate with the least number of votes (with tie-breaking) is eliminated, and their votes are transferred. The process continues until N_k candidates have been selected, or the number of remaining candidates equals the number of remaining seats. STV is commonly used in practice in multi-winner elections.

Details can differ in how such votes are transferred, but our results in Section 3 do not depend on the transfer rule; in our simulations in Section 4 and Appendix B, we use the “fractional” STV rule, in which each voter keeps a fractional number of votes equal to their share of the winning candidate’s surplus votes (we describe this formally in Section 4). For consistency, for all rules we assume that ties are broken in favor of candidates from party D, but randomly within each party.

[Next verbatim source excerpt]

At a high level, the proof establishes that no candidate receives meaningful (in terms of party vote share) votes from voters of the other party; in terms of party vote share, we can proceed with running two separate STV processes. Then, the sequential round path dependence does not matter, as the overall number of first-place votes for candidates of each party is invariant, and so the final partisan outcome depends just on initial vote shares. We note that the proof does not depend on our assumption of fractional STV; other transfer rules (such as random voter transfer) that preserve the number of surplus votes would induce seat shares according to the same formula.

[Next verbatim source excerpt]

A Thiele rule is characterized by a function λ . Each voter v in district k provides a set S_v of the N_k candidates they like most. The set of winners is determined as follows. Consider a potential set of winners C . The amount of points that voter v contributes to C is $\sum_{i=1}^{|S_v \cap C|} \lambda(i)$, and

the set C with the most points across voters (after tie-breaking) is selected. The (non-increasing) function λ establishes that votes have diminishing returns; the i th approved candidate in the set is worth $\lambda(i)$. For example, winner-take-all approval voting is a Thiele rule with $\lambda(i) = 1$: for each voter, the set C receives a number of points equal to the number of candidates in the set that the voter likes, and so the N_k candidates with the most votes win. We further consider Proportional Approval Voting (PAV), a Thiele rule with $\lambda_{PAV}(i) = \frac{1}{i}$; and Thiele Squared, with $\lambda_{TS}(i) = \frac{1}{i^2}$. PAV is well-studied in the theoretical social choice literature and comes with optimality guarantees (in terms of proportionality) (Skowron 2021). Thiele Squared induces even sharper diminishing returns than PAV in a voter having more of their preferred candidates winning, and thus prioritizes more voters having at least one approved candidate (and 50/50 outcomes between two parties, regardless of underlying vote shares). More generally, Thiele rules both contain common voting rules and parameterize a rule’s tendency to favor proportional or balanced outcomes.

[Next verbatim source excerpt]

In Single Transferable Vote (STV), a type of ranked choice voting for multiple winners, each voter instead submits a ranking over candidates (here, we assume the rankings are complete, not partial). Consider a district with N_k seats and V_k voters; the set of winners is created iteratively, as follows. The number of first-place votes for each candidate is counted. Any candidate with a number of votes at least the “Droop” threshold $Q = \lfloor \frac{V_k}{N_k + 1} \rfloor + 1$ is selected as a winner, and their surplus votes (number of votes minus Q) are transferred to each voter’s next preference. If no candidate has enough votes, then the candidate with the least number of votes (with tie-breaking) is eliminated, and their votes are transferred. The process continues until N_k candidates have been selected, or the number of remaining candidates equals the number of remaining seats. STV is commonly used in practice in multi-winner elections.

Details can differ in how such votes are transferred, but our results in Section 3 do not depend on the transfer rule; in our simulations in Section 4 and Appendix B, we use the “fractional” STV rule, in which each voter keeps a fractional number of votes equal to their share of the winning candidate’s surplus votes (we describe this formally in Section 4). For consistency, for all rules we assume that ties are broken in favor of candidates from party D, but randomly within each party.

Expanded PaperInterface specification

```

V {Voter : Type u_1} {Candidate : Type u_2} {partyVoters otherPartyVoters allVoters : Finset Voter}
{ballots : Voter → AppliedModelingLib.SocialChoice.Voting.Ballot Candidate}
{partyCandidates otherPartyCandidates initialActive : Finset Candidate} {seats voters : ℕ} {partyShare : ℝ},
1 ≤ seats →
0 ≤ partyShare →
partyShare ≤ 1 →
seats * (seats + 1) ≤ voters →
seats ≤ partyCandidates.card →
seats ≤ otherPartyCandidates.card →
GGRS26CombatingGerrymanderingRCV.paper_complete_party_ranking_ballots partyVoters ballots partyCandidates
initialActive →
```

```

GGRS26CombatingGerrymanderingRCV.paper_complete_party_ranking_ballots otherPartyVoters ballots
  otherPartyCandidates initialActive →
  voters = allVoters.card →
  allVoters = partyVoters ∪ otherPartyVoters →
  Disjoint partyVoters otherPartyVoters →
  Disjoint partyCandidates otherPartyCandidates →
  partyCandidates ⊆ initialActive →
  otherPartyCandidates ⊆ initialActive →
  initialActive ⊆ partyCandidates ∪ otherPartyCandidates →
  partyShare * ↑ voters = ↑ partyVoters.card →
  (1 - partyShare) * ↑ voters = ↑ otherPartyVoters.card →
  (∀
    (policy :
      GGRS26CombatingGerrymanderingRCV.BallotRoutedSTVTransferPolicy allVoters
        ballots ↑ (AppliedModelingLib.SocialChoice.Voting.STVQuota seats voters)),
    (∃ (terminal :
      GGRS26CombatingGerrymanderingRCV.BallotRoutedSTVState allVoters
        initialActive),
      GGRS26CombatingGerrymanderingRCV.BallotRoutedDFavoringSTVRun ballots
        (↑ (AppliedModelingLib.SocialChoice.Voting.STVQuota seats voters)) policy
        seats
        (GGRS26CombatingGerrymanderingRCV.paper_proposition1_ballot_routed_stv_initial_state
          allVoters initialActive)
        terminal ∧
        GGRS26CombatingGerrymanderingRCV.BallotRoutedSTVTerminal seats
        terminal) ∧
      ∀
        (terminal :
          GGRS26CombatingGerrymanderingRCV.BallotRoutedSTVState allVoters
            initialActive),
          GGRS26CombatingGerrymanderingRCV.BallotRoutedDFavoringSTVRun ballots
            (↑ (AppliedModelingLib.SocialChoice.Voting.STVQuota seats voters)) policy
            seats
            (GGRS26CombatingGerrymanderingRCV.paper_proposition1_ballot_routed_stv_initial_state
              allVoters initialActive)
            terminal →
          GGRS26CombatingGerrymanderingRCV.BallotRoutedSTVTerminal seats terminal →
          GGRS26CombatingGerrymanderingRCV.paper_seat_share_rounded
            (GGRS26CombatingGerrymanderingRCV.ballotRoutedPartyFinalSeats
              partyCandidates seats terminal)
            partyShare seats) ∧
          GGRS26CombatingGerrymanderingRCV.paper_seat_share_rounded
            (GGRS26CombatingGerrymanderingRCV.paper_pav_selected_seat_count partyShare
              seats)
            partyShare seats
    )
  )

```

Retained governing declarations

- [AppliedModelingLib.SocialChoice.Voting.Ballot](#)
- [AppliedModelingLib.SocialChoice.Voting.STVQuota](#)
- [GGRS26CombatingGerrymanderingRCV.BallotRoutedDFavoringSTVRun](#)
- [GGRS26CombatingGerrymanderingRCV.BallotRoutedSTVState](#)
- [GGRS26CombatingGerrymanderingRCV.BallotRoutedSTVTerminal](#)
- [GGRS26CombatingGerrymanderingRCV.BallotRoutedSTVTransferPolicy](#)
- [GGRS26CombatingGerrymanderingRCV.ballotRoutedPartyFinalSeats](#)
- [GGRS26CombatingGerrymanderingRCV.paper_complete_party_ranking_ballots](#)
- [GGRS26CombatingGerrymanderingRCV.paper_pav_selected_seat_count](#)
- [GGRS26CombatingGerrymanderingRCV.paper_proposition1_ballot_routed_stv_initial_state](#)
- [GGRS26CombatingGerrymanderingRCV.paper_seat_share_rounded](#)

Lean-elaborated claim roles

1. parameter: Type u_1
2. parameter: Type u_2
3. parameter: Finset Voter
4. parameter: Finset Voter
5. parameter: Finset Voter
6. parameter: Voter → AppliedModelingLib.SocialChoice.Voting.Ballot Candidate
7. parameter: Finset Candidate
8. parameter: Finset Candidate
9. parameter: Finset Candidate
10. parameter: ℕ
11. parameter: ℕ
12. parameter: ℝ
13. assumption: 1 ≤ seats
14. assumption: 0 ≤ partyShare
15. assumption: partyShare ≤ 1
16. assumption: seats * (seats + 1) ≤ voters
17. assumption: seats ≤ partyCandidates.card
18. assumption: seats ≤ otherPartyCandidates.card

19. assumption: GGRS26CombatingGerrymanderingRCV.paper_complete_party_ranking_ballots partyVoters ballots partyCandidates initialActive
 20. assumption: GGRS26CombatingGerrymanderingRCV.paper_complete_party_ranking_ballots otherPartyVoters ballots otherPartyCandidates initialActive
 21. assumption: voters = allVoters.card
 22. assumption: allVoters = partyVoters ∪ otherPartyVoters
 23. assumption: Disjoint partyVoters otherPartyVoters
 24. assumption: Disjoint partyCandidates otherPartyCandidates
 25. assumption: partyCandidates ⊆ initialActive
 26. assumption: otherPartyCandidates ⊆ initialActive
 27. assumption: initialActive ⊆ partyCandidates ∪ otherPartyCandidates
 28. assumption: partyShare * ↑ voters = ↑ partyVoters.card
 29. assumption: (1 - partyShare) * ↑ voters = ↑ otherPartyVoters.card
 30. conclusion: (∀
 (policy :
 GGRS26CombatingGerrymanderingRCV.BallotRoutedSTVTransferPolicy allVoters ballots
 ↑ (AppliedModelingLib.SocialChoice.Voting.STVQuota seats voters))),
 (∃ (terminal : GGRS26CombatingGerrymanderingRCV.BallotRoutedSTVState allVoters initialActive),
 GGRS26CombatingGerrymanderingRCV.BallotRoutedDFavoringSTVRun ballots
 (↑ (AppliedModelingLib.SocialChoice.Voting.STVQuota seats voters)) policy seats
 (GGRS26CombatingGerrymanderingRCV.paper_proposition1_ballot_routed_stv_initial_state allVoters
 initialActive)
 terminal ∧
 GGRS26CombatingGerrymanderingRCV.BallotRoutedSTVTerminal seats terminal) ∧
 ∀ (terminal : GGRS26CombatingGerrymanderingRCV.BallotRoutedSTVState allVoters initialActive),
 GGRS26CombatingGerrymanderingRCV.BallotRoutedDFavoringSTVRun ballots
 (↑ (AppliedModelingLib.SocialChoice.Voting.STVQuota seats voters)) policy seats
 (GGRS26CombatingGerrymanderingRCV.paper_proposition1_ballot_routed_stv_initial_state allVoters
 initialActive)
 terminal →
 GGRS26CombatingGerrymanderingRCV.BallotRoutedSTVTerminal seats terminal →
 GGRS26CombatingGerrymanderingRCV.paper_seat_share_rounded
 (GGRS26CombatingGerrymanderingRCV.ballotRoutedPartyFinalSeats partyCandidates seats terminal) partyShare
 seats) ∧
 GGRS26CombatingGerrymanderingRCV.paper_seat_share_rounded
 (GGRS26CombatingGerrymanderingRCV.paper_pav_selected_seat_count partyShare seats) partyShare seats

Lean proof endpoint (built separately)

GGRS26CombatingGerrymanderingRCV.paper_proposition1_source_selected_stv_and_pav

Recorded source-to-Spec screening Verdict: matches

Reason: The target retains $V \geq M(M+1)$, the two-party solid-coalition model, at least M candidates per party, exact Droop quota, D-favoring STV and the floor-or-ceiling STV/PAV conclusion; within-party randomness is used only as pathwise outcome support, not as a probability-law claim.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation